



สถาบันวิทยาการหุ่นยนต์ภาคสนาม (FIBO)
Institute of field Robotics

Final Presentation

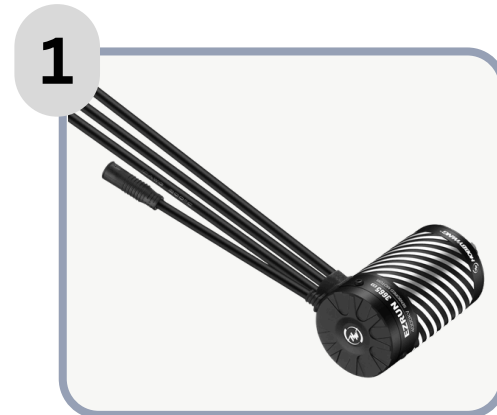
F1TENTH

6619 6631 6637 6644 6677

Component & Specification

Dimension

- BLDC controls via UART Protocol
- Servo for steering controls via micro-ROS (STM32G474RE)
- Estimate runtime for 2 Lipo 4s battery at 2 hours and 30 minutes.



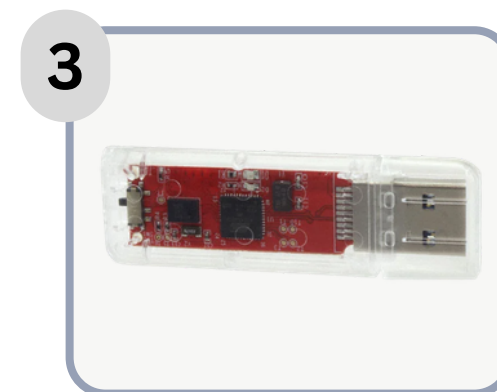
3665 G3 2400KV BLDC

- KV Rating: 2400 KV
- Size: 3665 can size (36 mm × 65 mm)
- Voltage: 2S–4S LiPo compatible
- Shaft Diameter: 5 mm



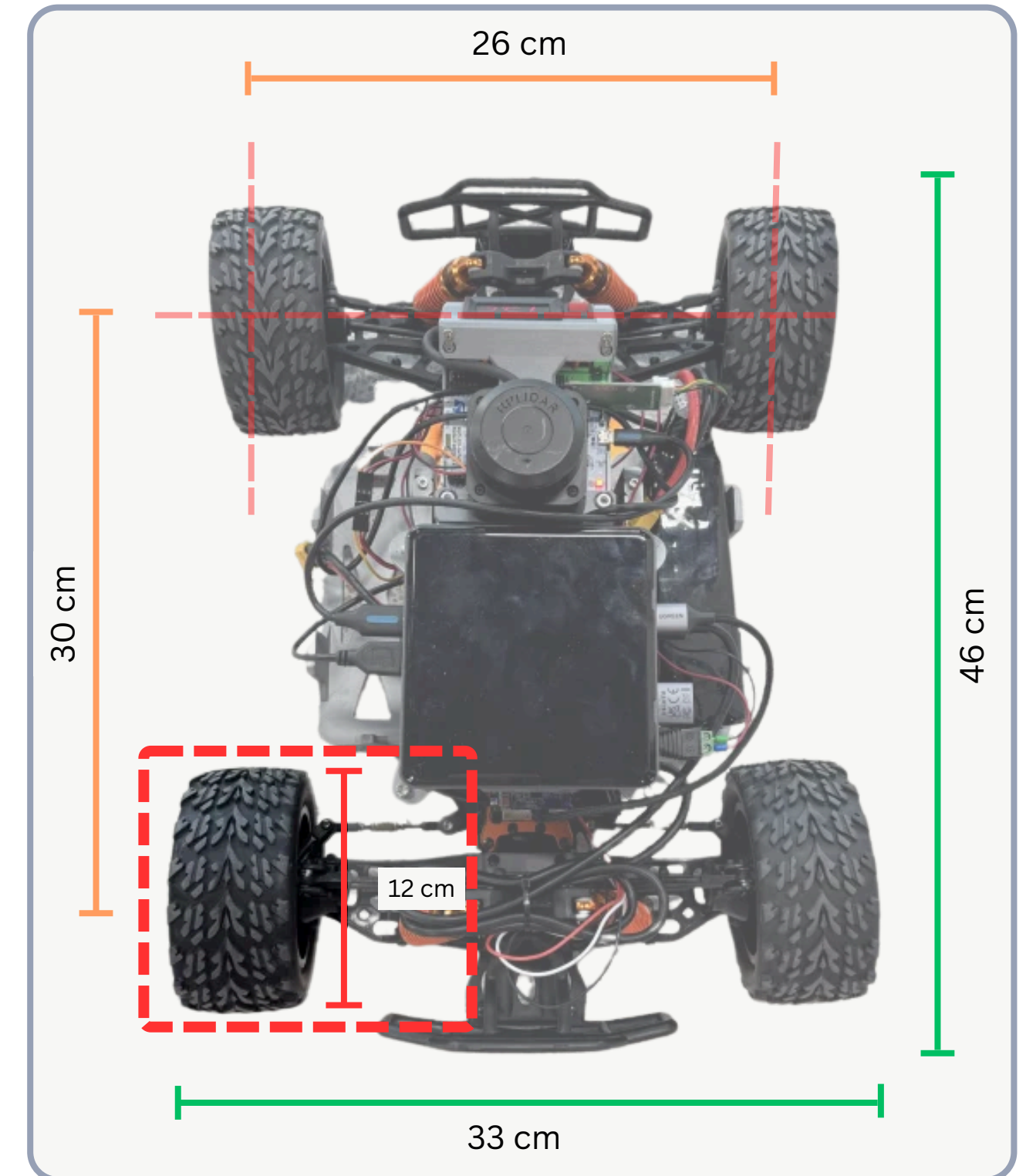
RPLIDAR C1

- Type: 2D 360° LiDAR scanner
- Range: Up to 12 m
- Scan Frequency: 5–10 Hz
- Angular Resolution: ~0.9°



BNO055 USB STICK

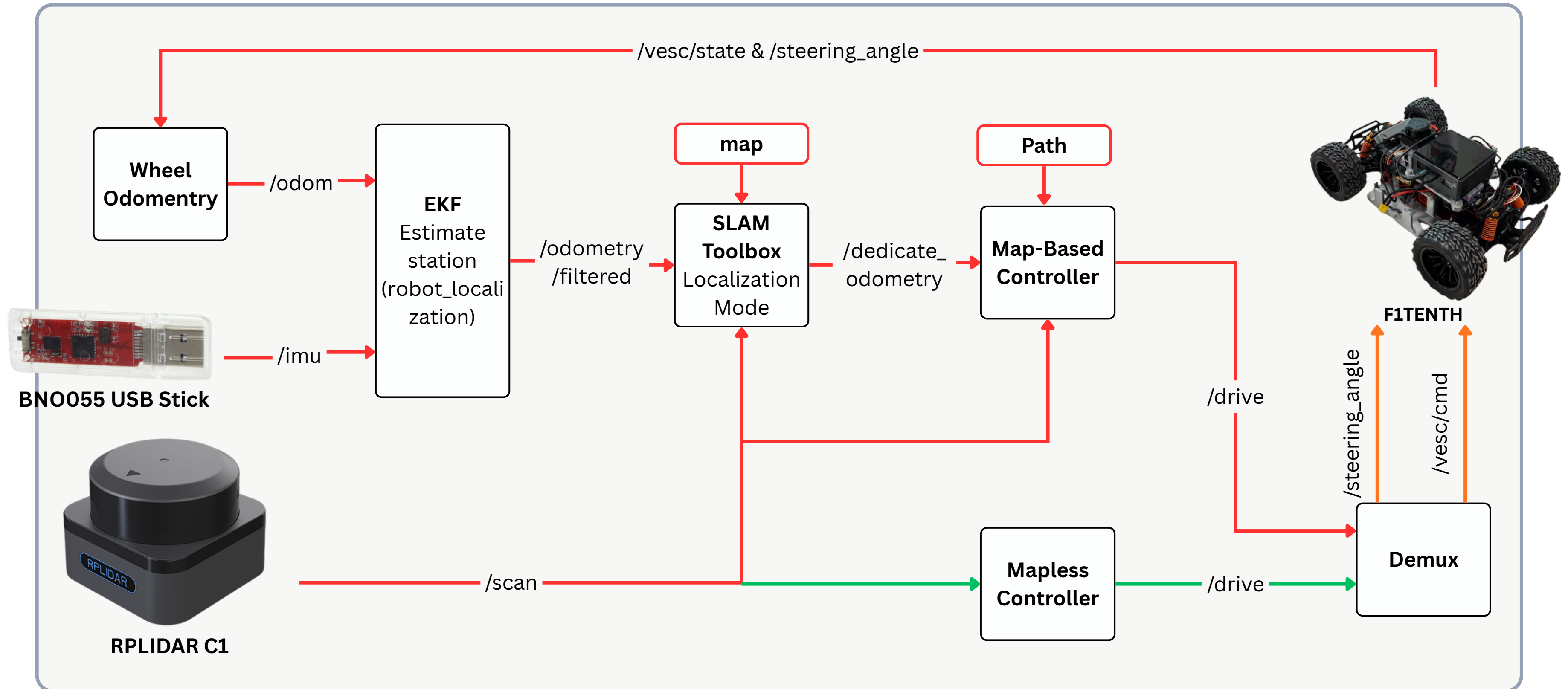
- Sensor Type: 9-DOF Absolute Orientation
- Output: Euler angles, Quaternion, Linear acceleration



Gear Ratio = 29.5 : 1

$\delta_{max} = \pm 24^\circ$

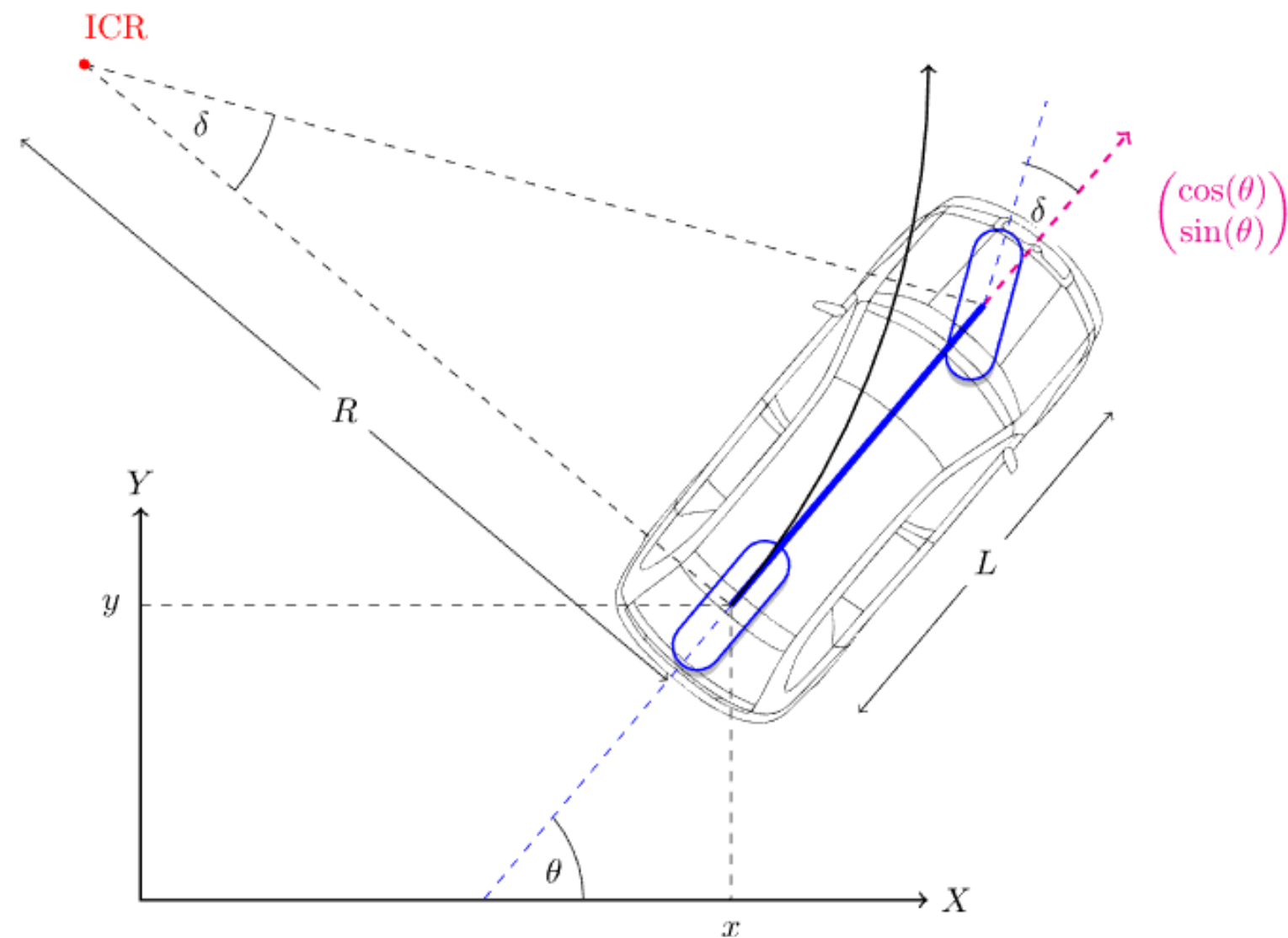
System Architecture



Bicycle Kinematic Model

To simplify 1 tenth into bicycle model

For the bicycle model, the **two front wheels** as well as the **two rear wheels** are **lumped into one wheel** each.

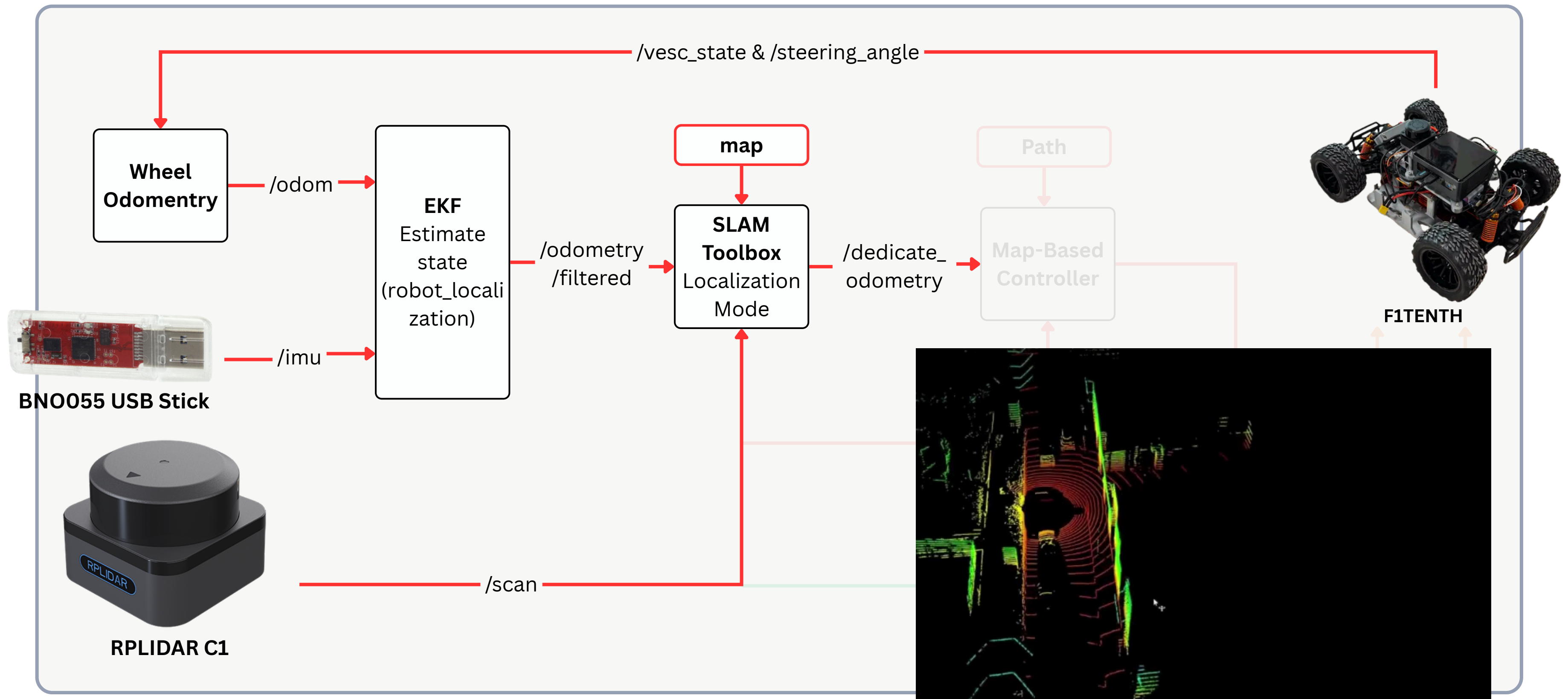


Using the **assumption** that there're **no slip in wheels** together with knowledge about the Instantaneous Center of Rotation (ICR), we can derive practical formulas

$$\dot{x} = v \cos(\theta), \quad \dot{y} = v \sin(\theta), \quad \dot{\theta} = \frac{v \tan(\delta)}{L}$$

$$\delta = \arctan \left(\frac{L \Omega_z}{v} \right) \text{ Steering Angle}$$

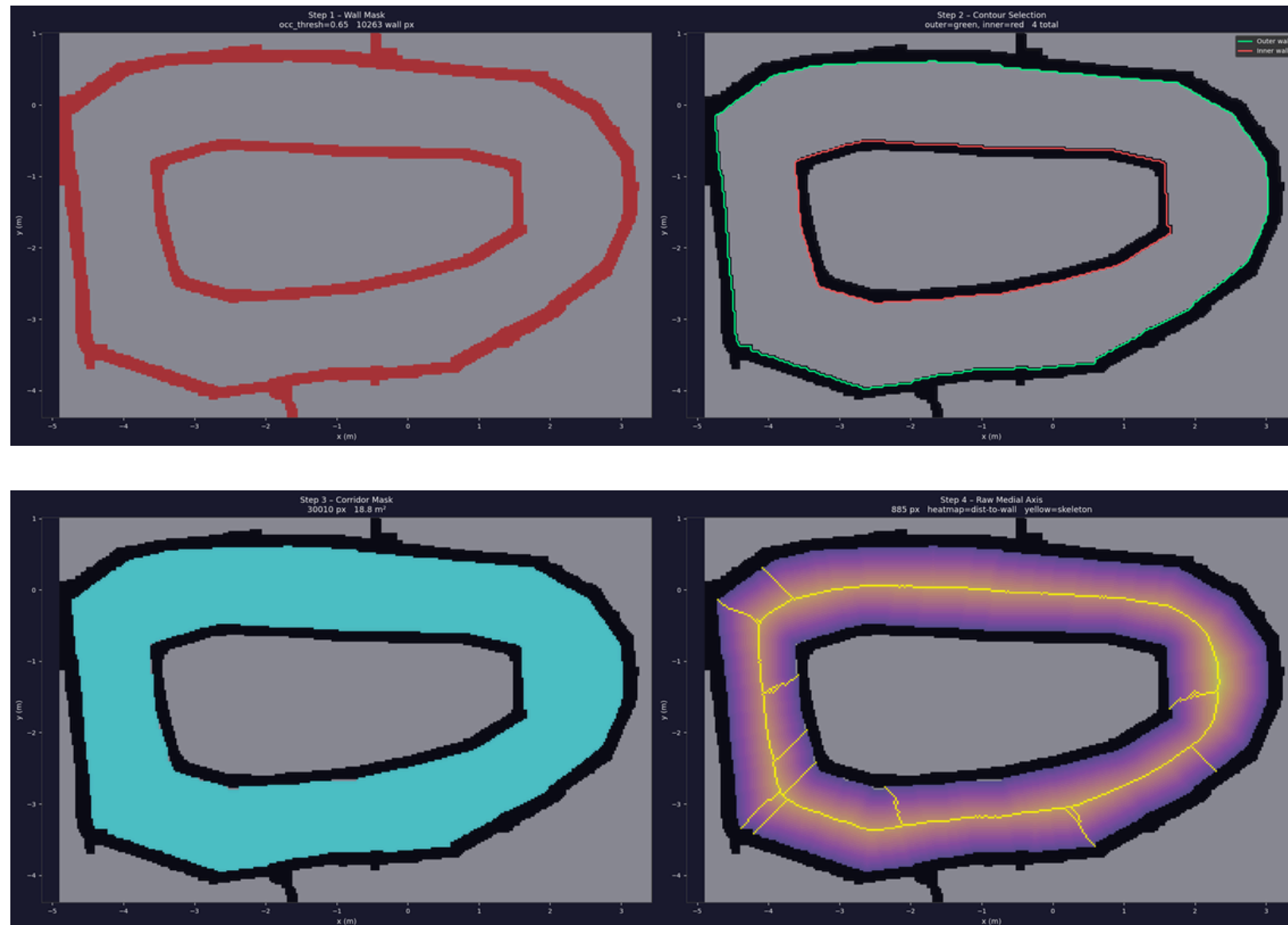
L = Wheel base



Raceline Generation

Map Loading Reads the .yaml map file to get resolution, origin, and image path.

Centerline Extraction



Step 1 – Wall Mask

Threshold the map image: pixels darker than $\text{occupied_thresh} \times 255$ are marked as walls. Dark = obstacle, bright = free space.

Step 2 – Contour Selection

Find all closed contours in the wall mask using a full hierarchy tree. Auto-select the outer track wall and the inner island wall.

Step 3 – Corridor Mask

Compute the driveable area: $\text{outer fill} - \text{inner fill} - \text{wall pixels}$. Produces a binary mask of the corridor the car can actually drive through.

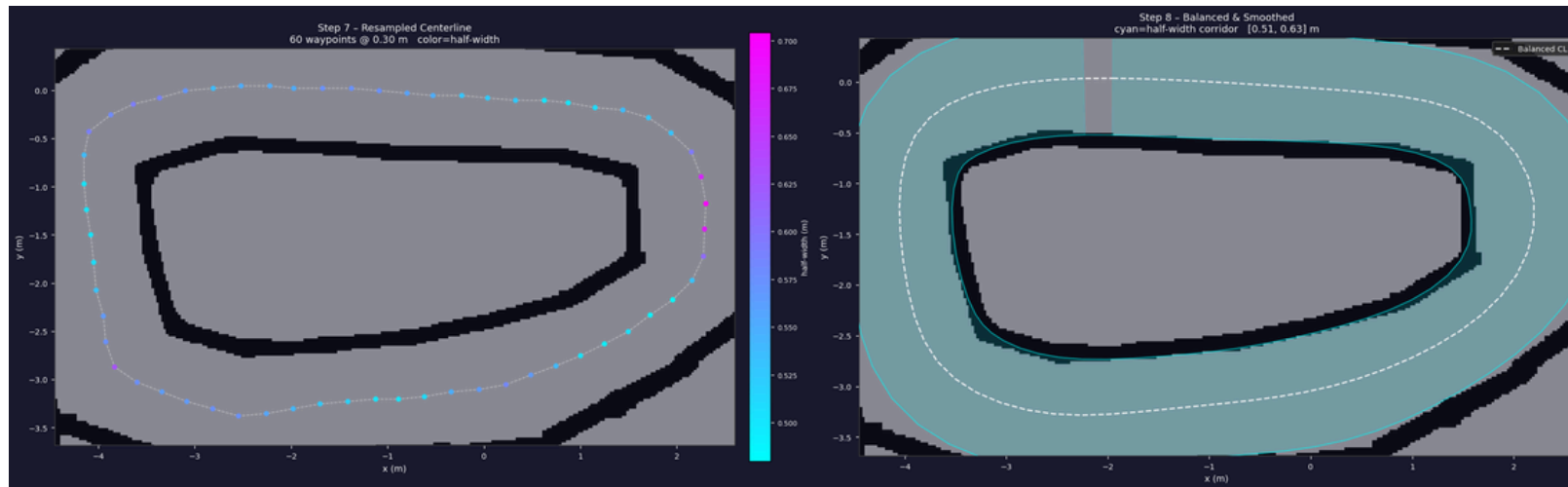
Step 4 – Raw Medial Axis

Run `skimage.medial_axis` on the corridor. The skeleton is the locus of all points equidistant from both walls.

Step 5 – Pruned Skeleton

Iteratively remove skeleton pixels with only one neighbor.

Centerline Extraction



Step 6 — Directed Main Loop

Build a graph where skeleton pixels are nodes and adjacency is edges. Use `networkx.cycle_basis` and pick the longest cycle — this is the closed track loop, with traversal direction preserved.

Step 7 — Resampled Centerline

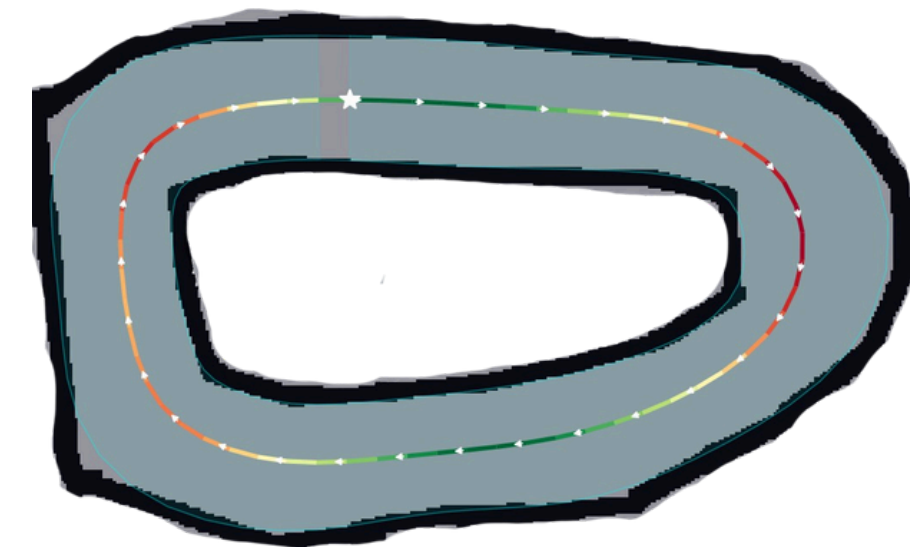
Arc-length parameterize the pixel loop, then re-interpolate at uniform spacing.

Step 8 — Balanced & Smoothed

Ray-cast perpendicular left and right from each waypoint, nudge toward the midpoint until $\text{gap}_L \approx \text{gap}_R$.

Speed Profile

- **Lateral limit:** $v_{\text{lat}}[i] = \sqrt{a_{\text{lat}} / \kappa[i]}$ — **slower in corners, faster on straights.**
- **Forward pass (acceleration):** caps speed so the car **can't be going faster than achievable** by accelerating from the previous waypoint: $v[i] \leq \sqrt{v[i-1]^2 + 2 \cdot a_{\text{lon}} \cdot ds}$.
- **Backward pass (braking):** caps speed so the **car can actually brake in time** for the next waypoint: $v[i] \leq \sqrt{v[i+1]^2 + 2 \cdot a_{\text{lon}} \cdot ds}$.
- Clamp to $[v_{\text{min}}, v_{\text{max}}]$.

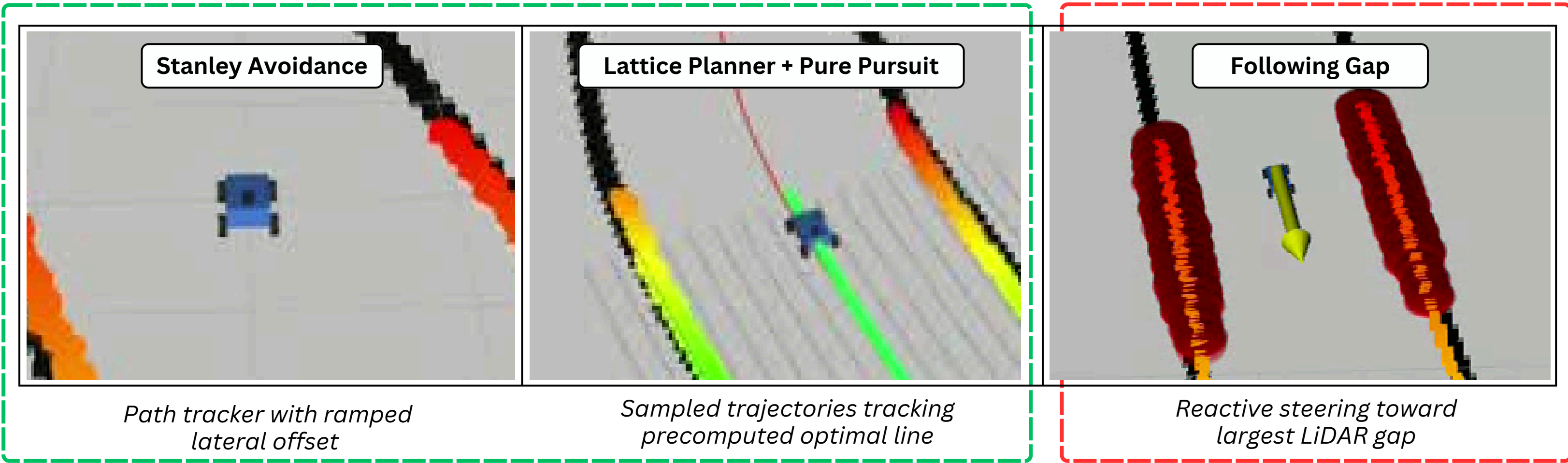


Controller Types

Main contibution

Local Planning / Obstacle Avoidance

Reactive Navigation



Map-based

Mapless

Baselines

Geometric Path Tracking



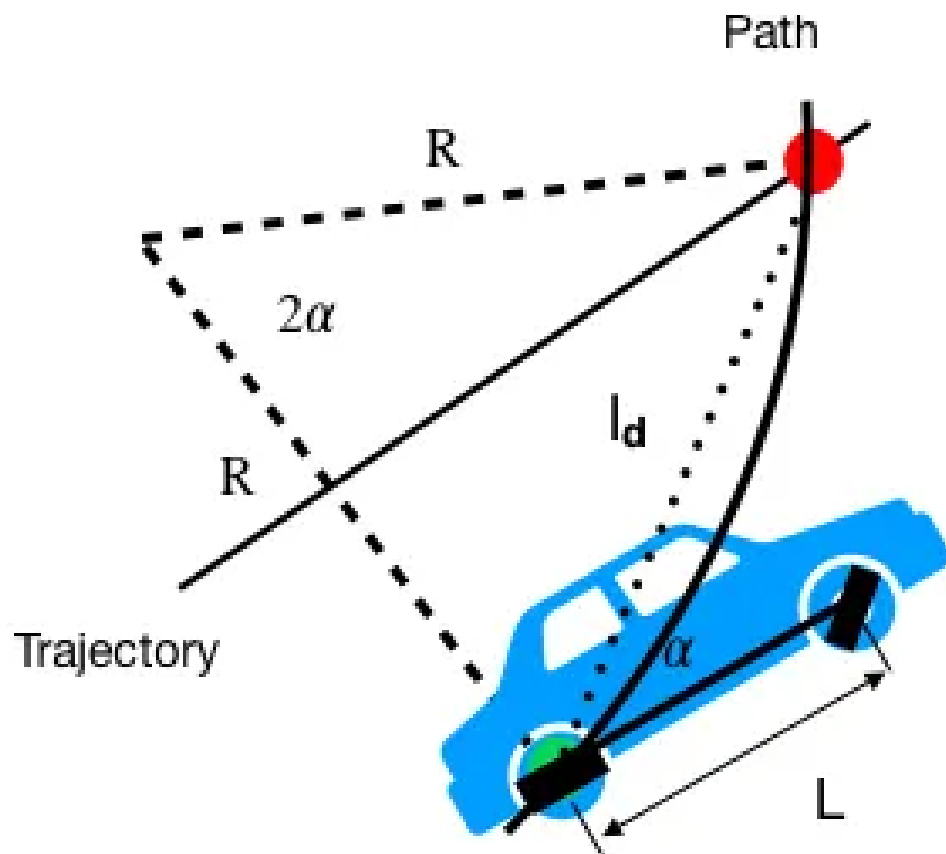
Stanley and Pure Pursuit serve as geometric-tracking baselines on the clean track

Stanley Avoidance, Lattice Raceline, and Follow the Gap are compared on the obstacle track, covering all three avoidance paradigms

Pure Pursuit Controller

What is Pure pursuit Controller ?

The Pure Pursuit controller is a geometric path tracking algorithm that determines the steering angle for a vehicle to reach a single point on a predefined path.



Look-Ahead Distance (LD)

The LD is the **fixed or dynamic distance** measured along the desired path from the vehicle's current position to a target point.

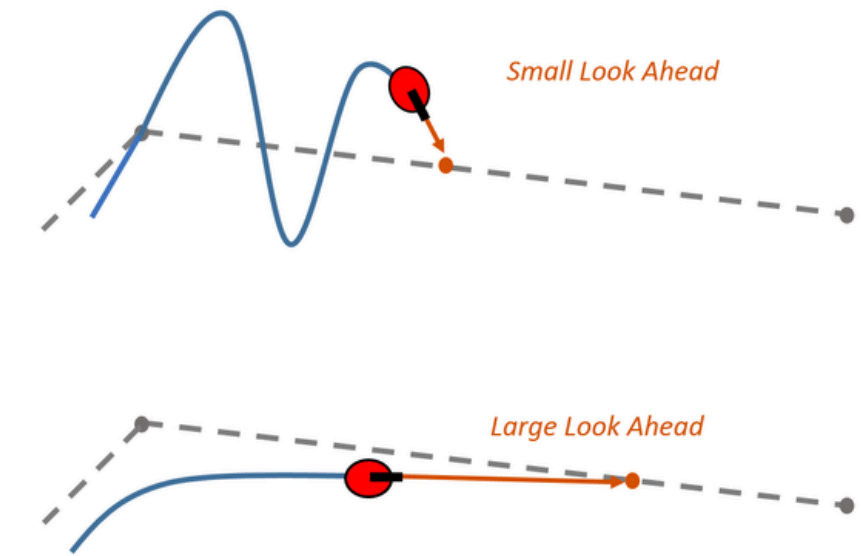
From geometric analysis of the circular arc, the instantaneous curvature is

$$k = \frac{1}{R} = \frac{2 \sin(\alpha)}{LD}$$

Dynamic Look-Ahead:

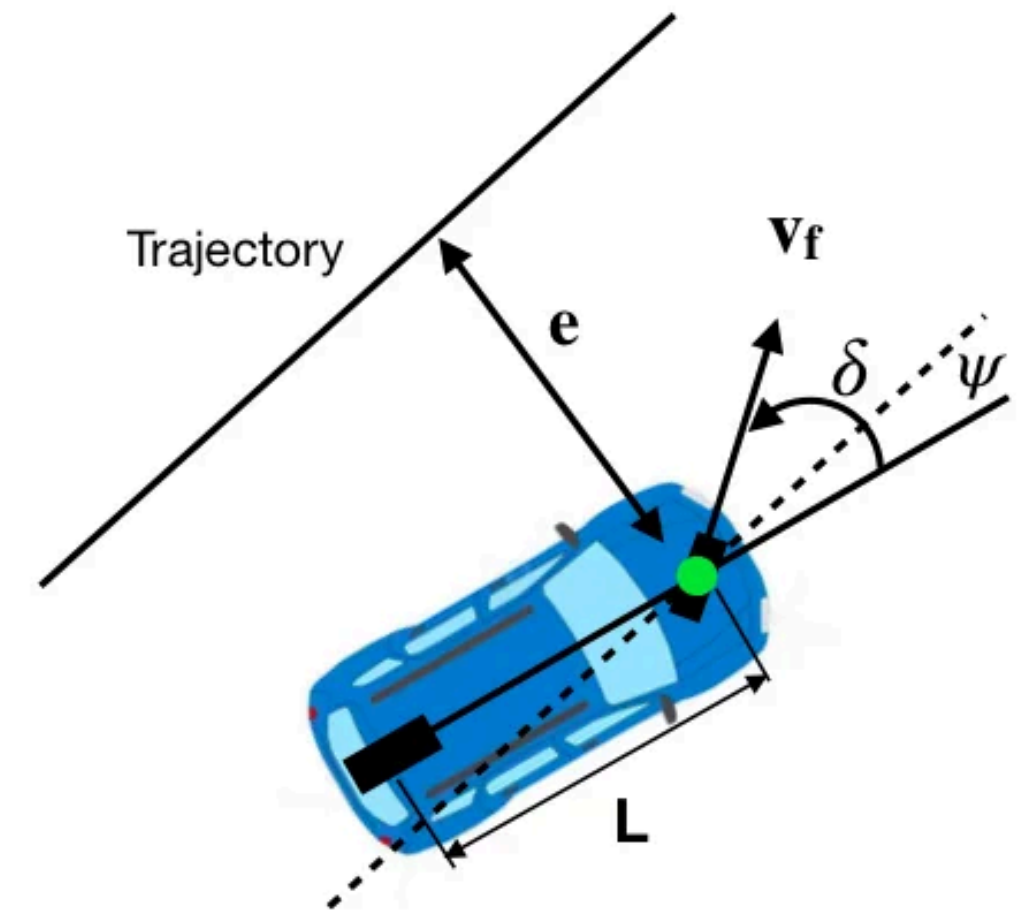
Kinematic
Bicycle
Model

$$\delta = \arctan\left(\frac{2 \sin(\alpha)}{LD}\right) = \arctan\left(\frac{2 \sin(\alpha)}{\min_{LD} + (K \times V_f)}\right)$$



What is Stanley Controller ?

- It is the path tracking approach, Stanley method use the front axle as its reference point. Meanwhile, it looks at both the **heading error** and **cross-track error**. In this method, the cross-track error is defined as the distance between the closest point on the path with the front axle of the vehicle.



Working Principle

$$\delta(t) = k_{\text{heading}} \psi(t) + \tan^{-1} \left(\frac{k_{\text{crosstrack}} e(t)}{v_f(t)} \right), \quad \delta(t) \in [\delta_{\min}, \delta_{\max}]$$

Steering Angle ← Heading Error ← Cross-tracking Error ← Velocity

Following Gap Mechanism

LiDAR scan Preprocessing

- LiDAR frame triggers the algorithm at 10 Hz.
- LiDAR point closer than 2.5 meters is treated as an obstacle
- Invalid LiDAR values are treated as far/free space.
- Only the front $\pm 120^\circ$ region is used for planning.

Obstacle inflation / safety bubble

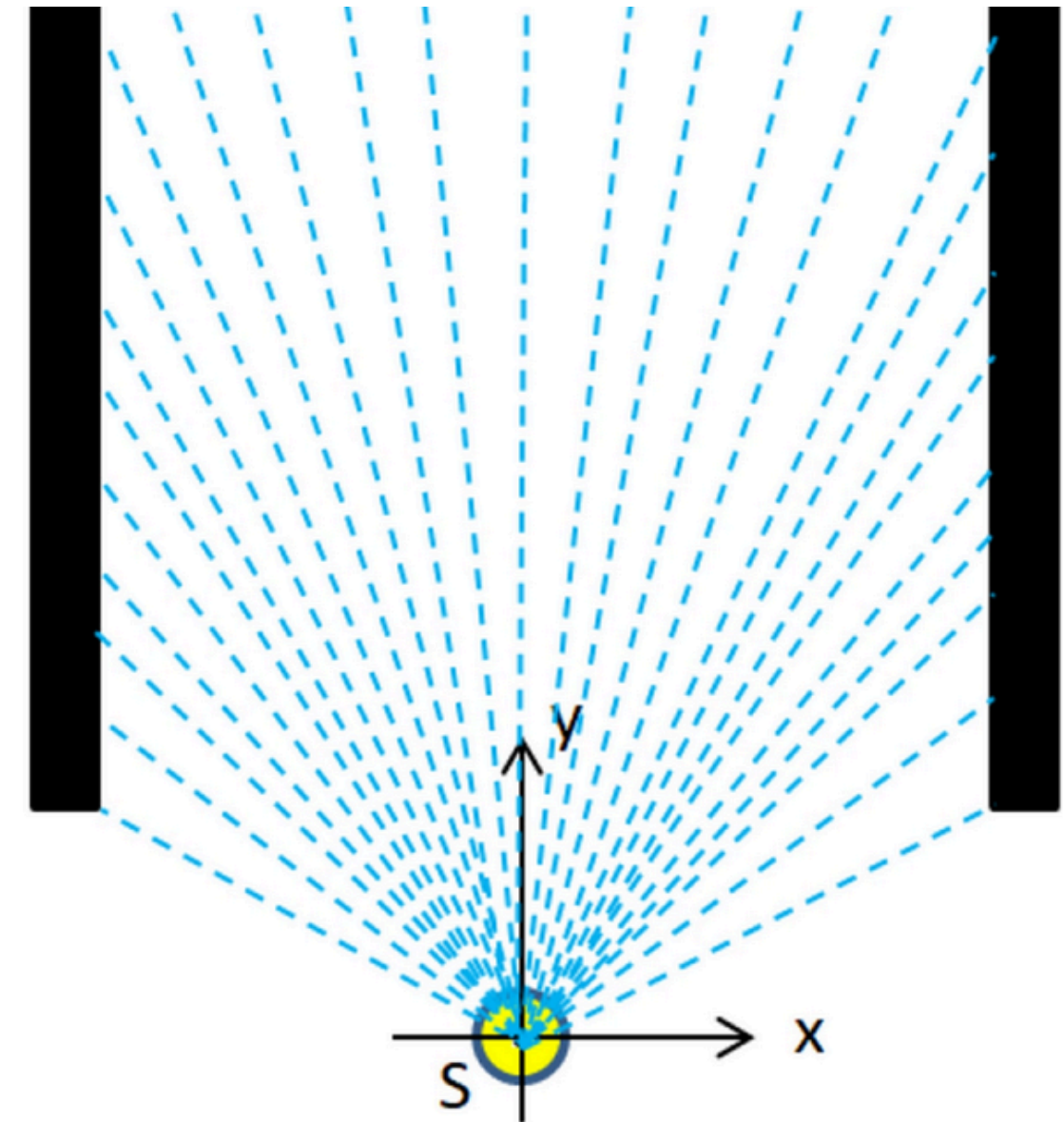
- Create adaptive safety bubble

Closer obstacle $\rightarrow$ bigger safety bubble

Farther obstacle $\rightarrow$ smaller safety bubble

- The physical bubble size is converted into LiDAR index width $B_i = \frac{b(\tilde{r}_i)}{\tilde{r}_i \Delta\theta}$
- Apply obstacle inflation to obstacle region

$$p_j = \begin{cases} 0, & j \in [i - B_i, i + B_i] \\ r_j, & \text{otherwise} \end{cases}$$



Following Gap Mechanism

Gap selection

1. Search continuous free gaps

- A gap is a continuous sequence of non-zero LiDAR ranges.

$p_j = 0 \Rightarrow$ unsafe direction

$p_j > 0 \Rightarrow$ candidate free direction

2. Measure how safe each gap

- Gap width: $W_k = e_k - s_k + 1$

LiDAR rays are inside this free region?

- Gap distance: $\min_{i \in G_k} p_i, \frac{1}{W_k} \sum_{i=s_k}^{e_k} p_i, \max_{i \in G_k} p_i$

Overall openness of the gap

- Gap Center: $c_k = \frac{s_k + e_k}{2}$

Is the gap in the center of the car $c_0 = \frac{N}{2}$

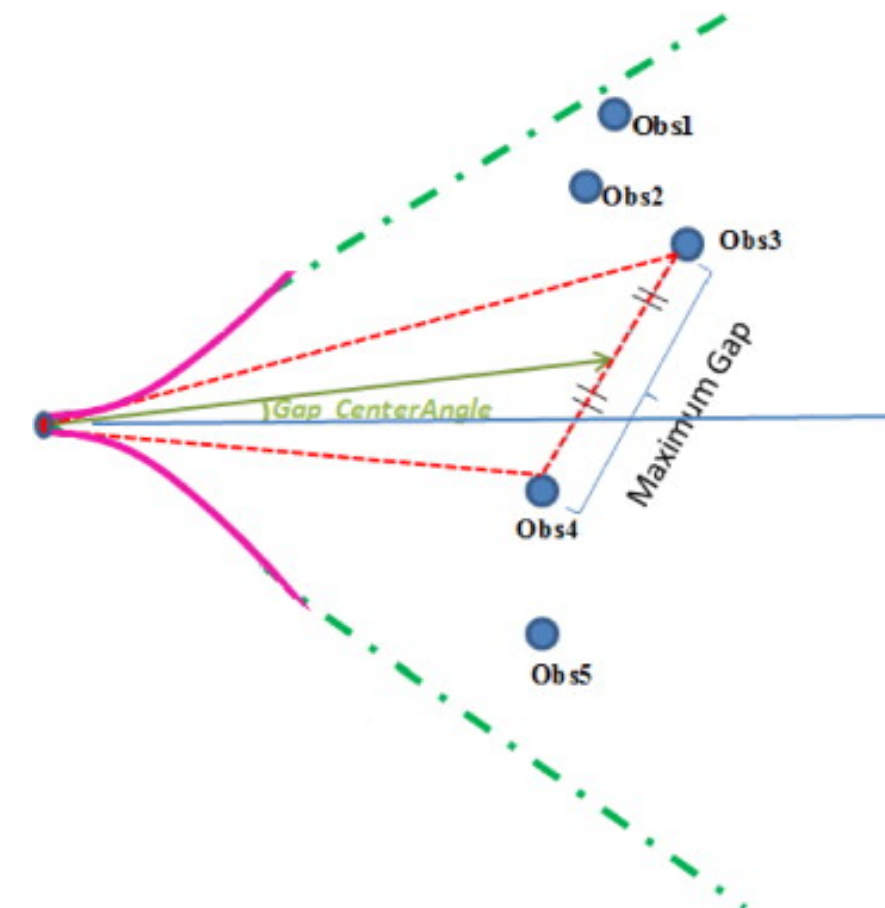
- Gap must be in the accepted range related to the car

3. Scoring the gap

$$S_{\text{center},k} = 1 - \left(\frac{|c_0 - c_k|}{c_0} \right) \quad S_k = 0.4d_{\text{max},k} + 0.6d_{\text{avg},k} \times S_{\text{center},k}$$

Best Gap = open and deep + not too far from center

- Algorithm prefers gaps that are generally open, not only gaps with one far point
- Best gap selection $k^* = \underset{k}{\operatorname{argmax}} S_k$



Following Gap Mechanism

Heading Selection

- The robot drives toward the center of the best gap:

$$c^* = \frac{s^* + e^*}{2}$$

$$\theta^* = \theta_{min} + c^* \Delta\theta$$

Clearance Estimation

The algorithm checks the minimum free distance around the turning direction:

$$D_{\text{clear}} = \min_{i \in \mathcal{W}(\theta^*)} p_i$$

- If turning right

$$\mathcal{W}(\theta^*) = [\theta^*, \theta^* + 45^\circ]$$

- If turning left:

$$\mathcal{W}(\theta^*) = [\theta^* - 45^\circ, \theta^*]$$

- If mostly straight:

$$\mathcal{W}(\theta^*) = [\theta^* - 30^\circ, \theta^* + 30^\circ]$$

Following Gap Mechanism

Speed control

- The raw speed depends on clearance:

$$v_{\text{raw}} = \begin{cases} v_{\text{min}}, & D_{\text{clear}} < 1.5 \\ v_{\text{max}}, & D_{\text{clear}} > 4.0 \\ K_p D_{\text{clear}}, & \text{otherwise} \end{cases}$$

- Then speed is reduced during sharp turns:

$$v = \text{clip}(v_{\text{raw}} (0.4 + 0.6 \cos |\theta^*|), v_{\text{min}}, v_{\text{max}})$$

Large Clearance + Small Steering Angle $\Rightarrow$ Higher Speed
Small Clearance + Sharp Turn $\Rightarrow$ Slower Speed

Steering control

$$\alpha_{\text{des}} = \text{clip}(\theta^*, -\alpha_{\text{max}}, \alpha_{\text{max}})$$

- The steering is smoothed to avoid sudden steering jumps.

$$\alpha_{\text{smooth}} = \lambda \alpha_{\text{des}} + (1 - \lambda) \alpha_{\text{prev}}$$

- steering is reduced when speed is high

$$f_v = 1 - 0.5 \left(\frac{v - v_{\text{min}}}{v_{\text{max}} - v_{\text{min}}} \right) \quad \alpha_{\text{cmd}} = f_v \alpha_{\text{smooth}}$$

The proposed system **performs obstacle avoidance while still using the Stanley controller** for path tracking. We controller temporarily **adds a lateral offset** to the Stanley cross-track error when an obstacle is detected.

Step 1 – LiDAR to 2D Occupancy Grid

Each LiDAR is converted to Cartesian space.

$$(i, j) = (-xC + H, yC + \frac{W}{2}), \quad x = r \cos \theta, \quad y = r \sin \theta$$

Step 2 – Path Inflation (Collision Checking)

Collision occurs if any occupied cell lies inside the inflated tube around the path segment.

$$r_{\text{inflate}} = r_{\text{vehicle}} + r_{\text{safety}}$$

$$\text{Collision} = \exists c \in (\text{PathSegment} \oplus r_{\text{inflate}}) : \text{Occ}(c) = 1$$

Step 3 – Obstacle Side & Target Offset

Side relative to path & Target offset magnitude from perpendicular distance

$$s = \text{sign}\left((\mathbf{p}_{\text{next}} - \mathbf{p}_{\text{prev}}) \times (\mathbf{p}_{\text{obs}} - \mathbf{p}_{\text{prev}})\right)$$

$$e_{\text{target}} = -s \cdot \text{clip}(d_{\perp} + r_{\text{vehicle}} + r_{\text{safety}}, e_{\text{min}}, e_{\text{max}})$$

Step 4 – Ramped Offset & Pass Detection

$$\Delta e = e_{\text{target}} - e_{\text{offset}}(k)$$

$$e_{\text{offset}}(k+1) = \begin{cases} e_{\text{target}}, & |\Delta e| \leq r \\ e_{\text{offset}}(k) + \text{sign}(\Delta e) \cdot r, & \text{otherwise} \end{cases}$$

$$e_{\text{target}} = 0 \quad \text{if } p_{\text{obs},x}^c < \tau_{\text{pass}}$$

$$\delta(t) = k_{\text{heading}} \psi(t) + \tan^{-1} \left(\frac{k_{\text{crosstrack}} (e(t) + e_{\text{offset}}(t))}{v_f(t)} \right)$$

Sense – build obstacle map

Lidar points are filtered to $\pm 135^\circ$ (front + sides). Each point is transformed from car frame to world frame, then split into two groups:

- **Obstacles** – points **within track_half_width** of the raceline $\rightarrow$ things to avoid
- **Walls** – points beyond that but within $+1.5$ m $\rightarrow$ track boundary

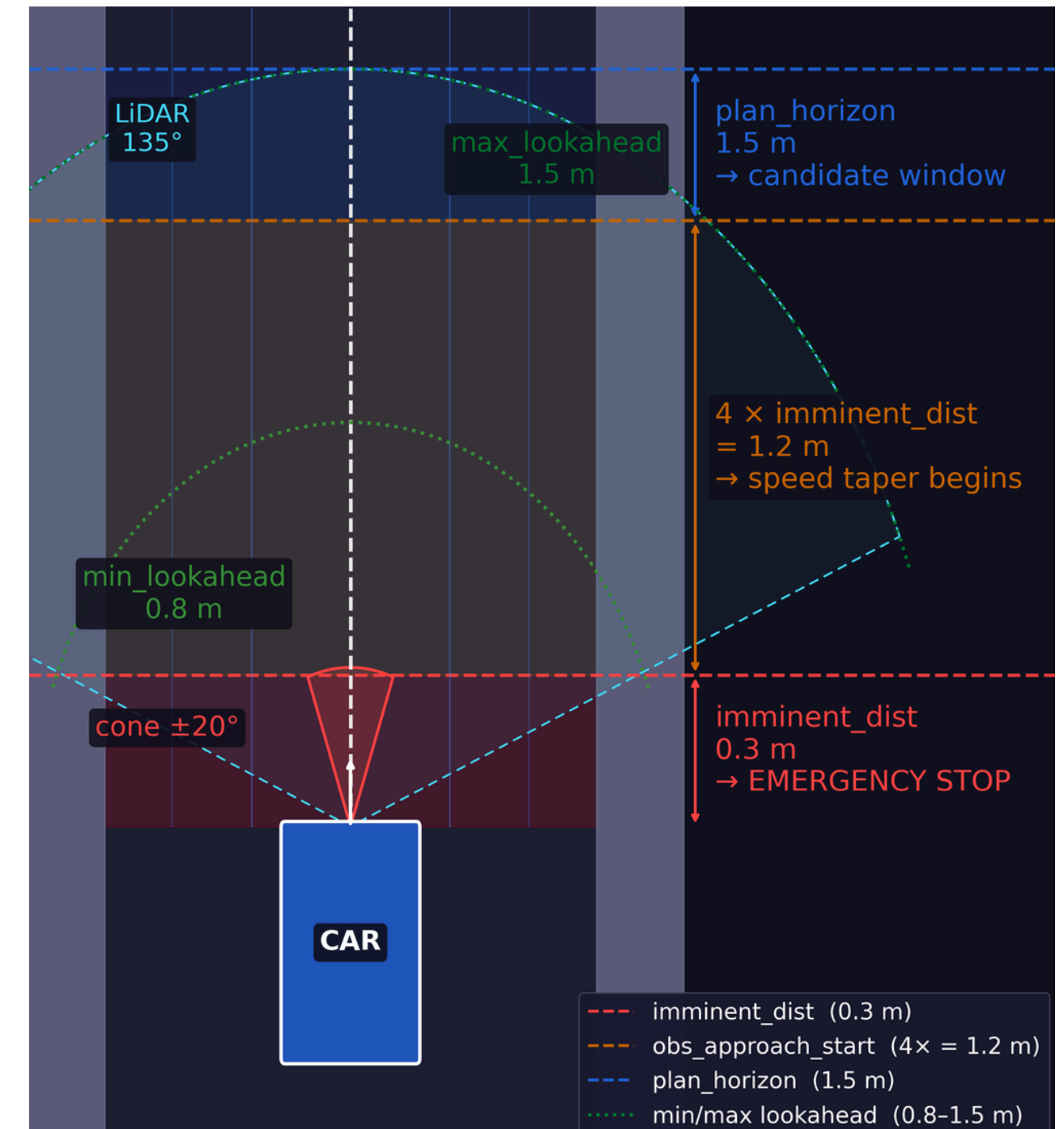
Points behind that range are ignored entirely – they're not on the track.

Plan – generate and score candidates

The planner generates candidate paths by shifting the entire window left or right along the perpendicular normals.

$$\begin{aligned} \text{score} = & w_{\text{deviation}} \times |\text{offset}| && \leftarrow \text{prefer staying near raceline} \\ & + 0.3 \times \text{curvature_cost} && \leftarrow \text{prefer smooth paths} \\ & + 25.0 \times \max(0, 0.8 - \text{surplus})^2 && \leftarrow \text{heavy penalty near obstacles} \\ & + 15.0 \times \text{wall_shortfall}^2 && \leftarrow \text{penalty for clipping walls} \\ & + w_{\text{continuity}} \times |\text{offset} - \text{prev_offset}| && \leftarrow \text{prefer not changing sides} \end{aligned}$$

Candidates that fall within **safety_radius** of any obstacle or wall are **hard rejected** before scoring.

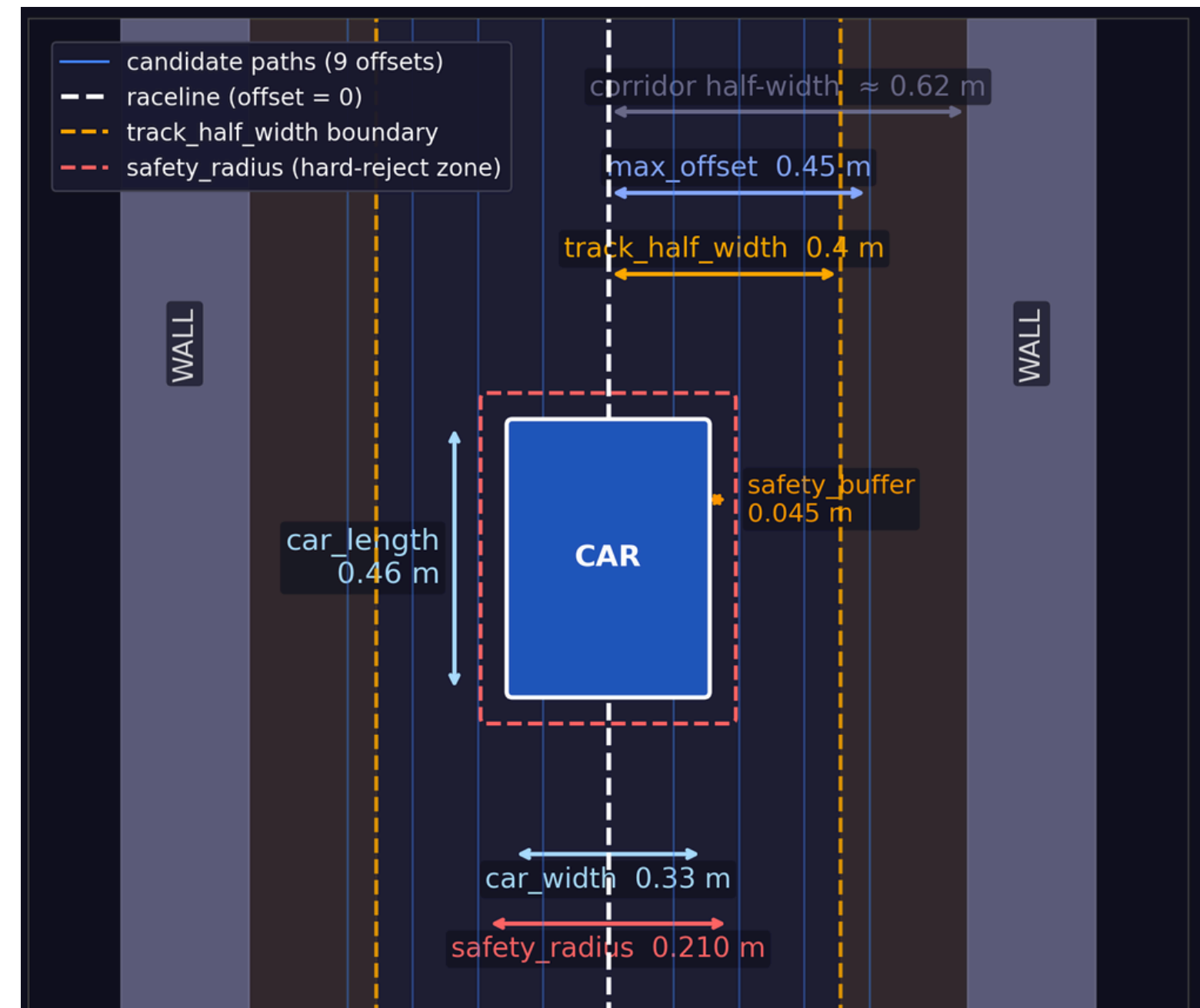


State machine – when to avoid

- **FOLLOWING:** **check if the centerline itself is blocked.** If yes, evaluate all candidates and lock onto the best offset.
- **AVOIDING:** **keep the committed path.** Re-plan only after `replan_hold_ticks` to prevent oscillation. Return to centerline only when both the committed path AND the centerline are clear for `clear_hold_ticks` consecutive ticks

Control – pure pursuit on selected path

- adaptive lookahead = speed × gain
- clamped to [0.8, 1.5] m → find target point → compute steering via bicycle model $\text{steer} = \text{atan}(L \times 2y / Ld^2)$





Field Setup

To evaluate all 5 different controller, we test them under different obstacles

Senario 1



Senario 2



Senario 3



- Racetrack width ranges from 120 to 160 cm
- Obstacle dimensions is not exceed $26 \times 18 \times 30$ cm
- All controllers are evaluated under identical track and speed conditions, each tuned to its own best-performing configuration.

Experiment 1 – Localization Accuracy

Purpose

Verify **SLAM Toolbox Localization Mode** produces stable, **non-diverging** pose across all operating conditions

Setup

- SLAM Toolbox **Localization Mode**, frozen map loaded
- **All controllers** run, **4 conditions** each
- **Same start pose** every trial

Tested Condition

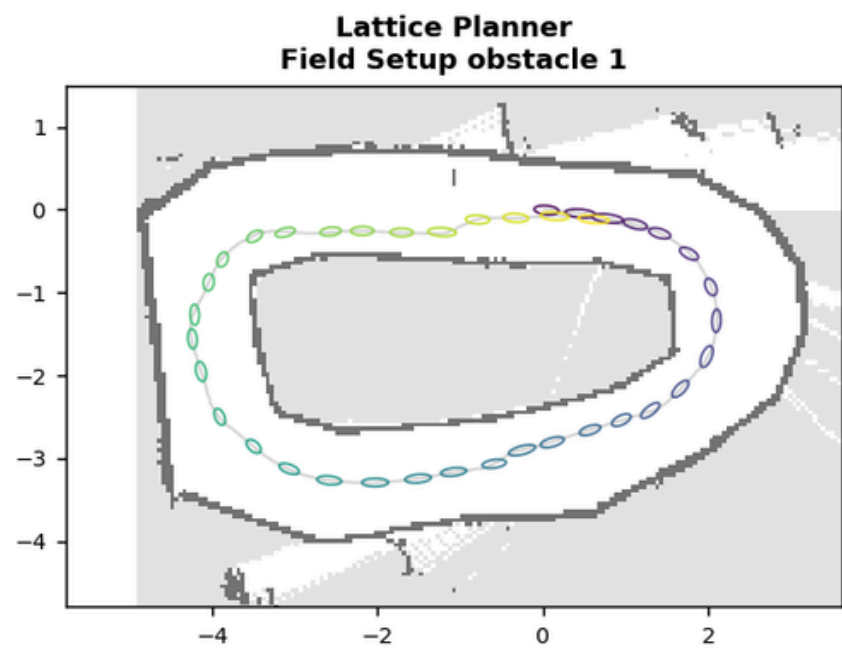
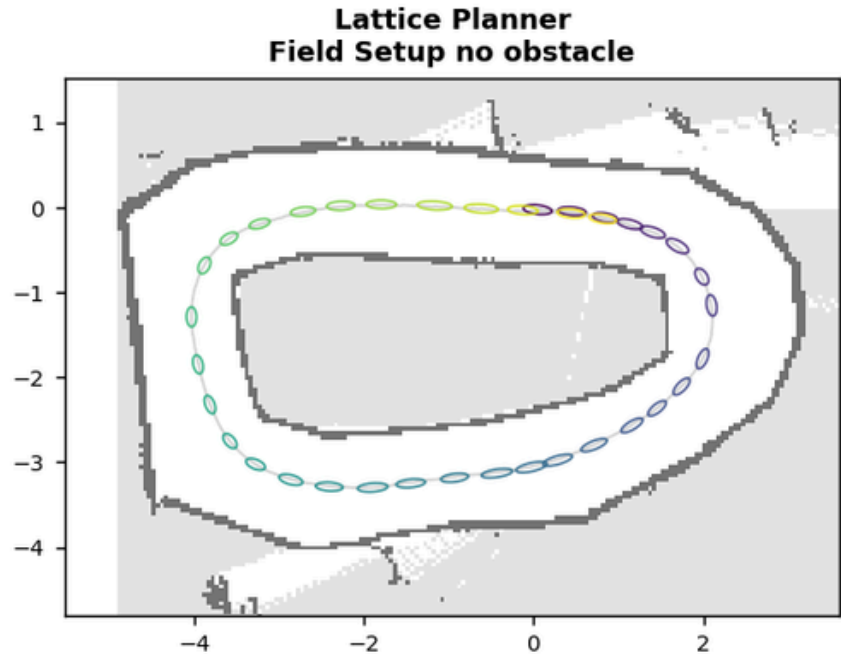
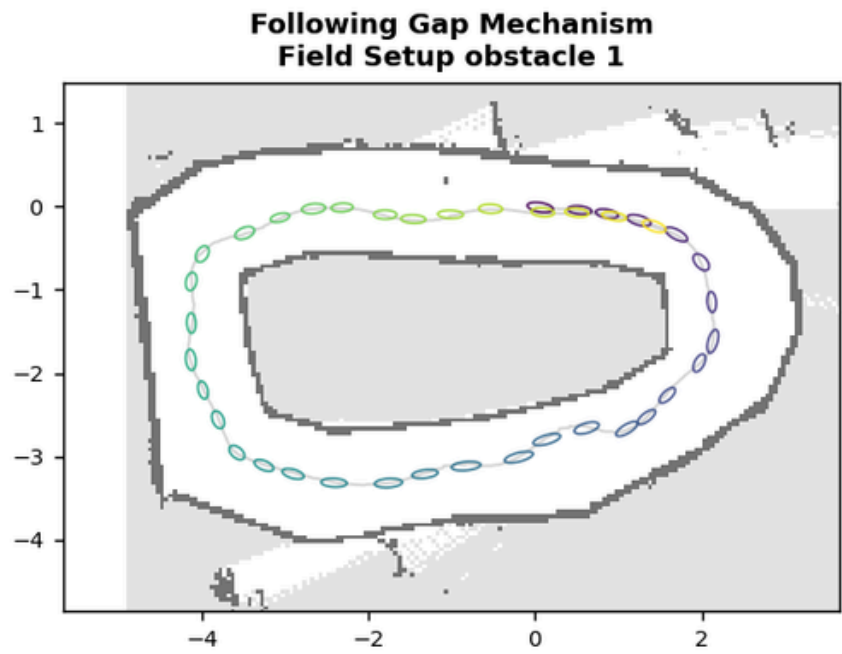
Condition	Description
No obs	Clean track, baseline
Scenario 1	Obstacle arrangement 1
Scenario 2	Obstacle arrangement 2
Scenario 3	Obstacle arrangement 3

Metrics

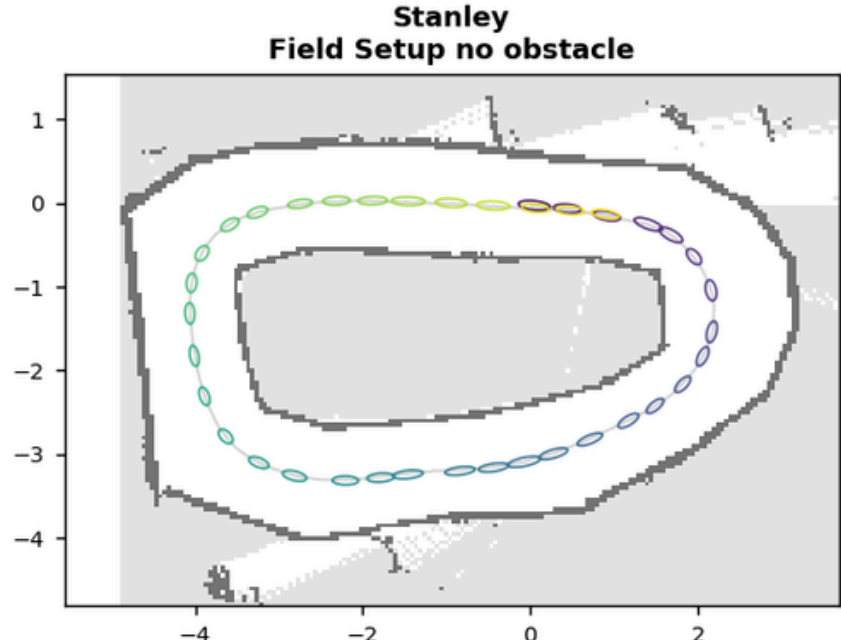
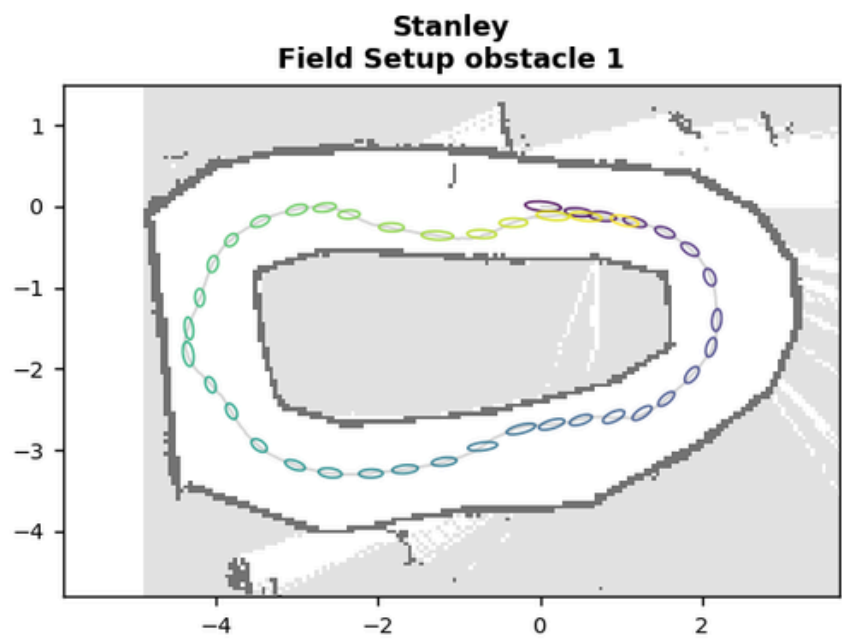
Metric	Reason
Covariance trace	Indicates filter confidence; divergence means localization lost
Pose jump rate	Large $ \Delta_{\text{pose}} $ between updates implies scan-match failure
Update rate	Must keep up for real-time control

Experiment 1 – Localization Accuracy

slam_toolbox — 5-experiment covariance ellipses



time within run (start → end)



Experiment	M1 max trace [m ²] (< 0.05)	M2 median Δxy [m] (≈ 0.085 m expected)	M3 odom rate [Hz] (≥ 20)	Verdict
Following Gap Mechanism — Field Setup obstacle 1	0.0087	0.0720	100	Pass
Lattice Planner — Field Setup no obstacle	0.0117	0.0990	95	Pass
Lattice Planner — Field Setup obstacle 1	0.0103	0.0970	98	Pass
Stanley — Field Setup obstacle 1	0.0131	0.0960	95	Pass
Stanley — Field Setup no obstacle	0.0114	0.0980	98	Pass

Experiment 2 – Basic Path Following

Purpose

Compare Pure Pursuit, Stanley, and Following Gap Mechanism on the raceline (**no obstacles**).

Setup

- **Pre-computed** raceline from frozen map
- **Localization** Mode active
- **Same start pose** every trial
- **Fixed speed** (single regime)

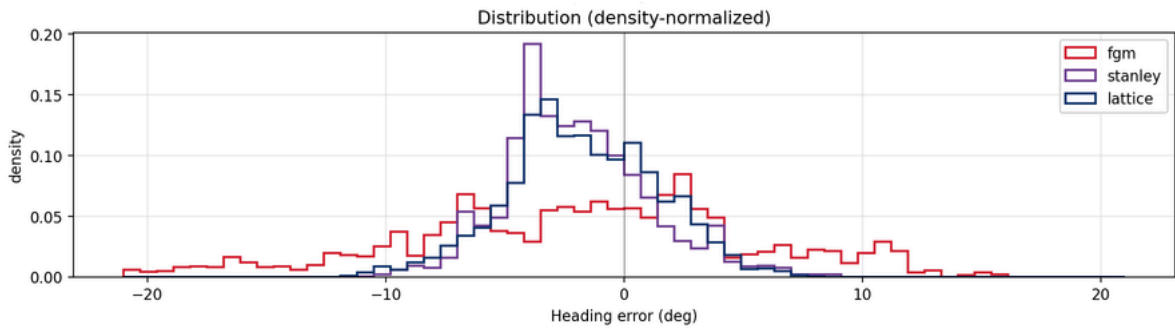
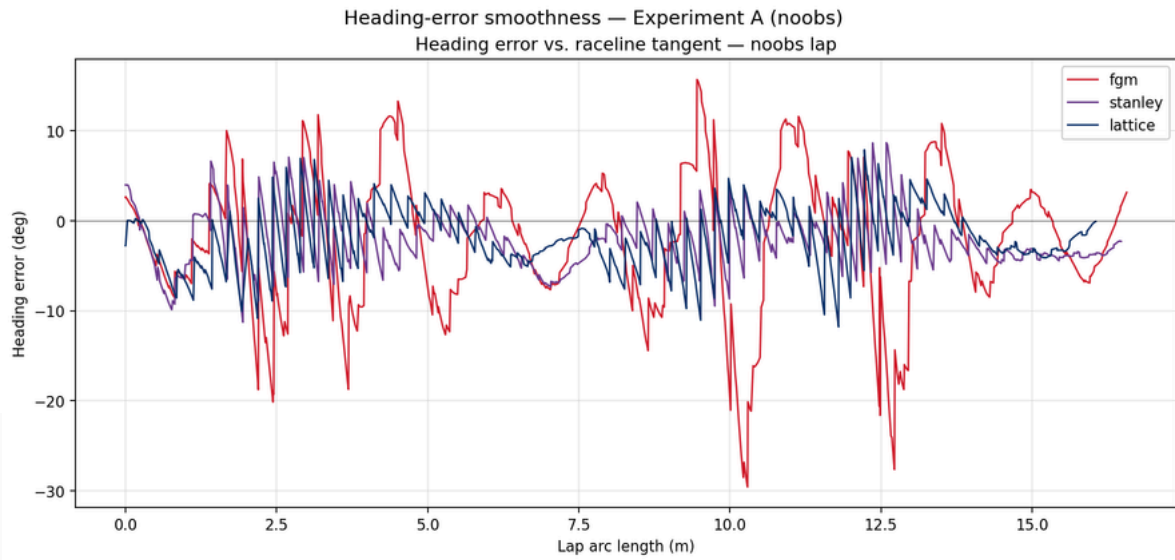
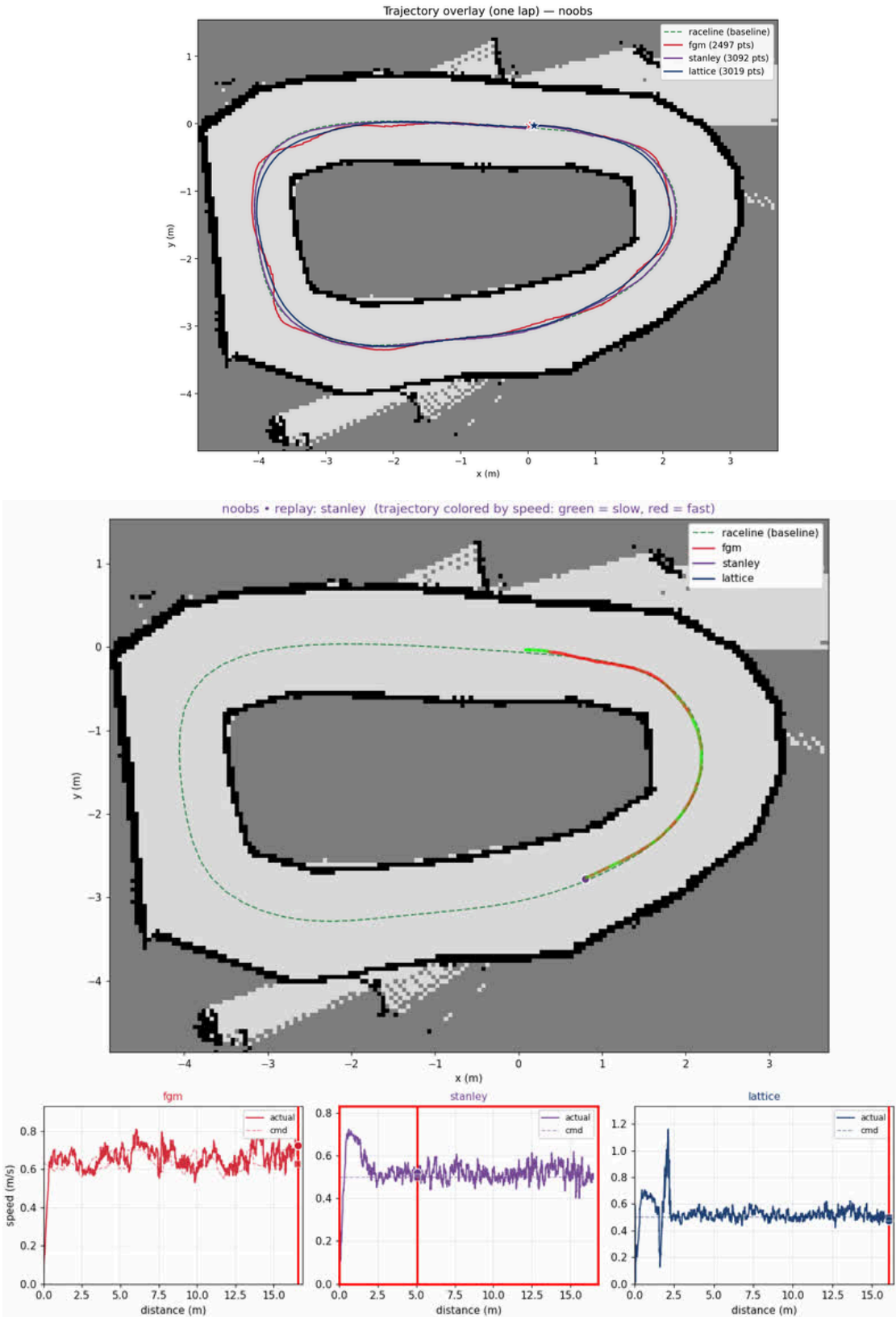
Tested Condition

Factor	Levels
Controller	Pure Pursuit, Stanley, FGM

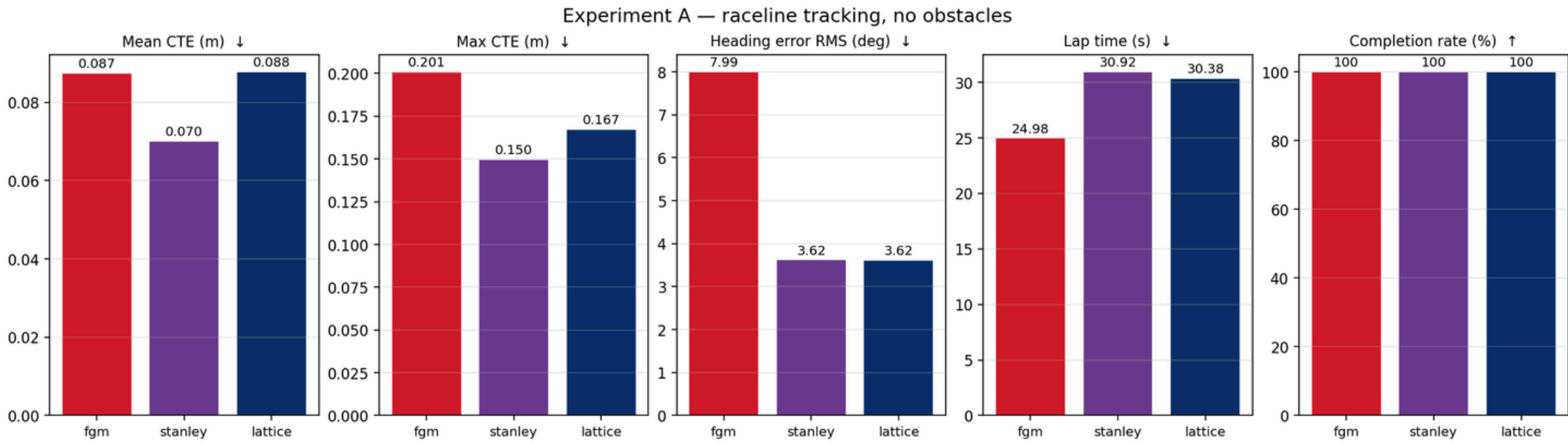
Metrics

Metric	Reason
Mean CTE	Average tracking accuracy along the lap
Max CTE	Worst-case deviation; safety indicator
Heading error RMS	Captures yaw misalignment Pure Pursuit cannot correct directly
Lap time	Performance metric for speed comparison
Completion rate	Reliability indicator

Experiment 2 – Basic Path Following



controller	mean (°)	RMS (°)	max err (°)	$\Delta err / sample \sigma$ (°)
fgm	-2.23	7.99	29.56	1.30
stanley	-1.99	3.62	11.26	1.05
lattice	-1.75	3.62	11.78	1.05



Purpose

Compare **Map-less** vs **Map-based avoidance** with **obstacles not in the map**.

Setup

- 3 Obstacle **Layouts**, **Obstacle** dimensions within **26 × 18 × 30 cm**
- **Same layouts** across all controllers
- Frozen map, **Localization Mode active**
- **Same start pose** every trial

Tested Condition

Factor	Levels
Controller	Map-less, Map-based
Layout	Scenario 1, 2, 3

Metrics

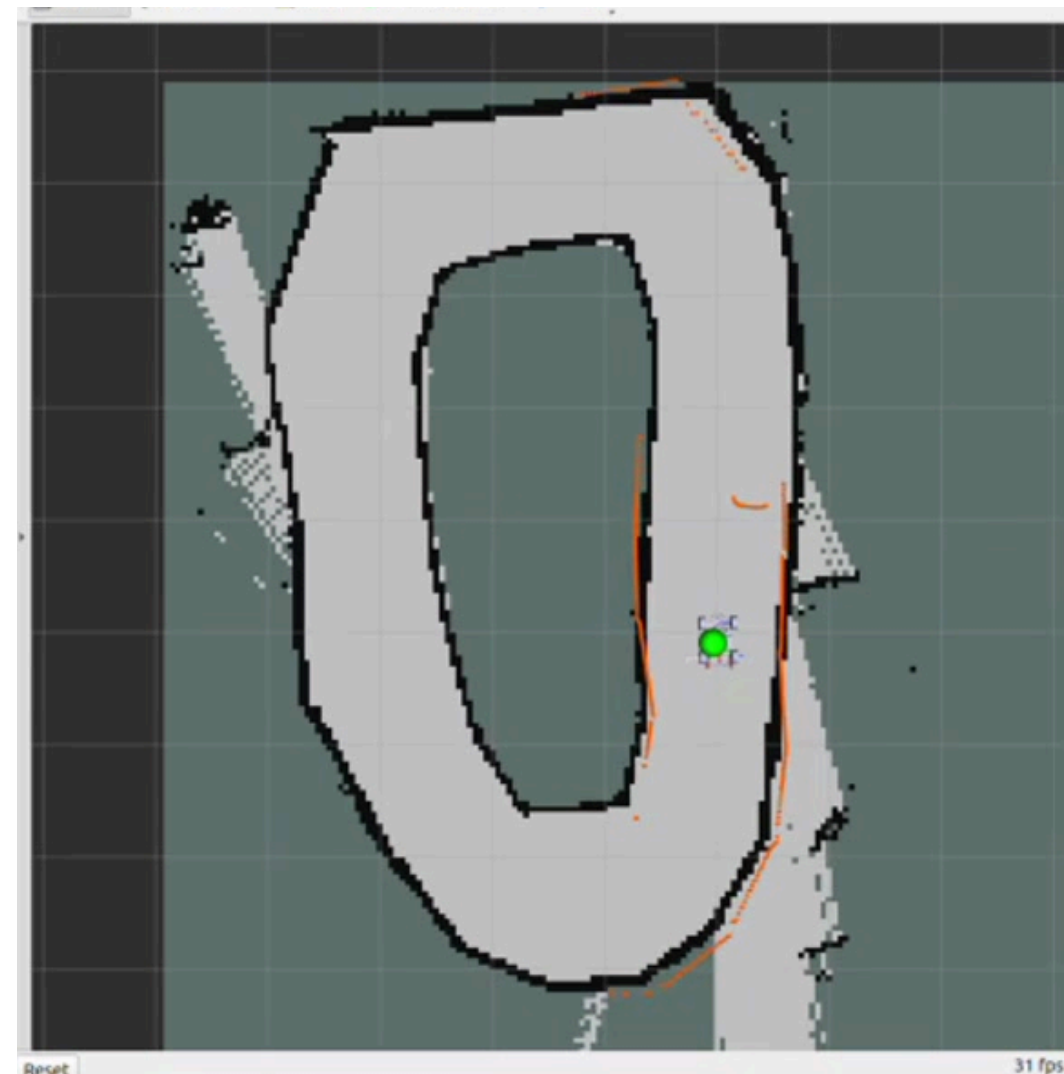
Metric	Reason
Success rate	Core safety indicator: avoidance worked AND lap finished
Min clearance	How close to collision; safety margin per controller
Mean CTE	Tracking quality during and after avoidance
Detour length	Path inefficiency cost of avoidance
Lap time	Speed penalty vs clean lap baseline (Exp 2)
Recovery distance	How quickly controller returns to raceline post-avoidance

Experiment 3 – Static Obstacle Avoidance

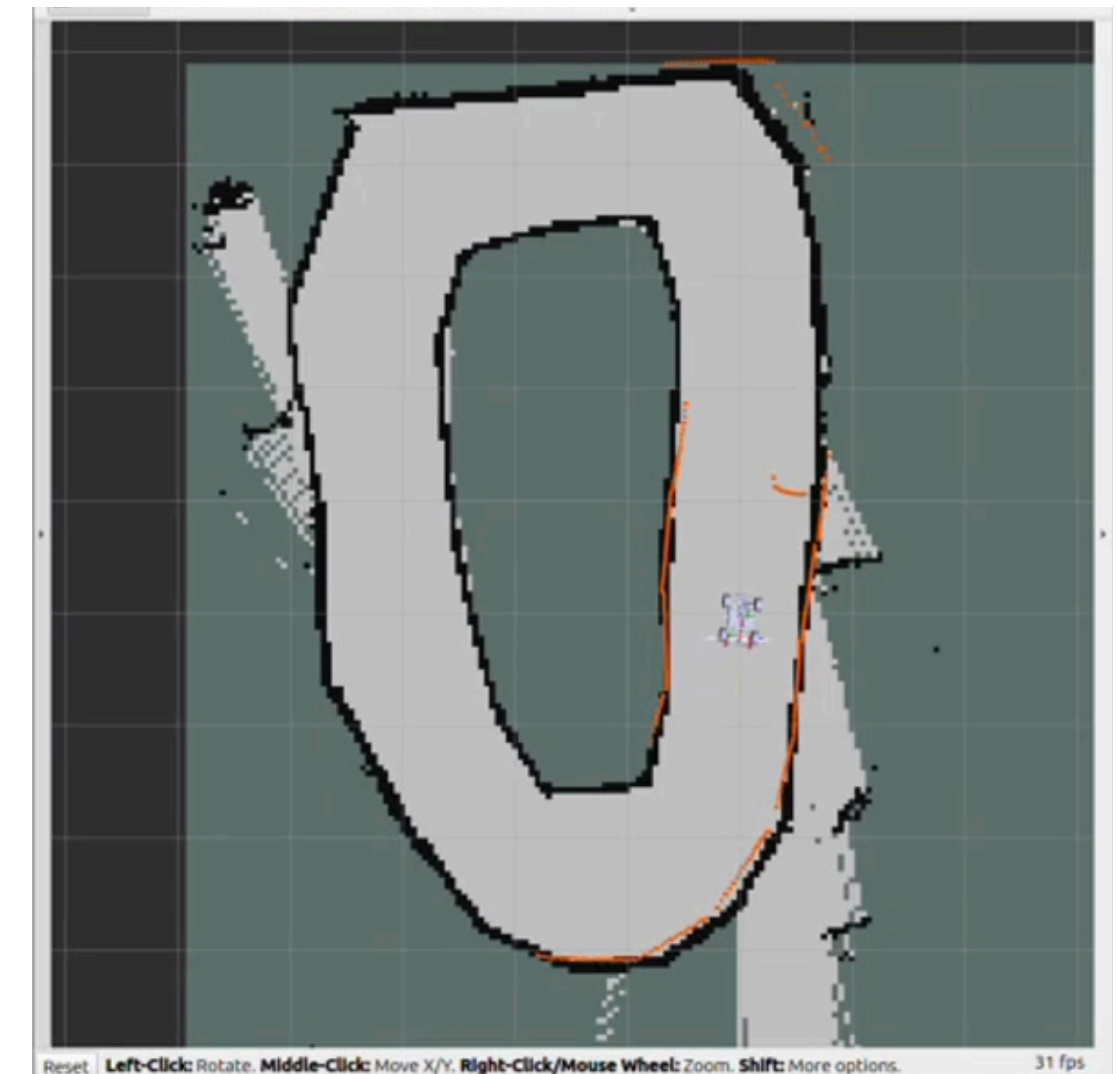
Following Gap Mechanism



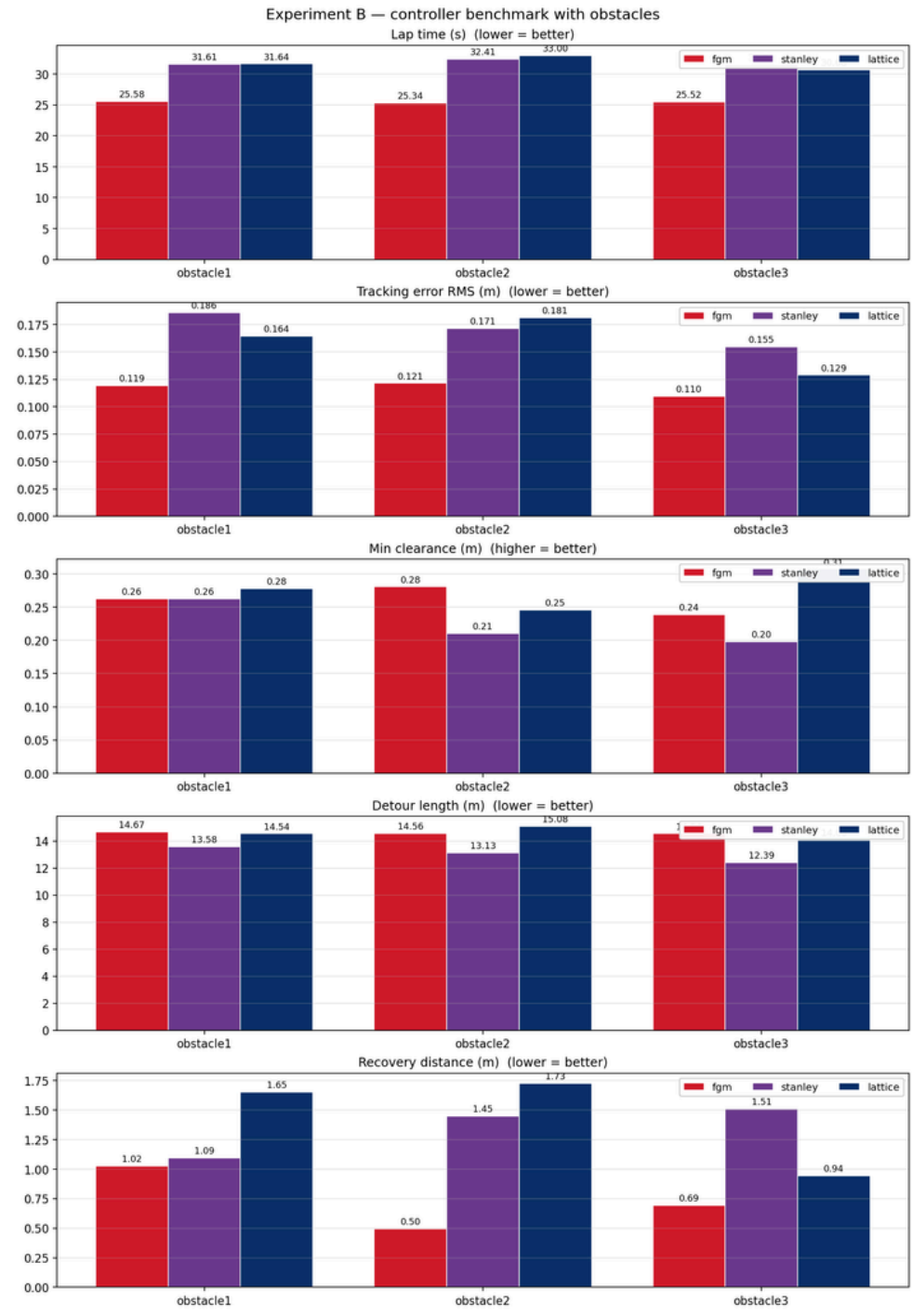
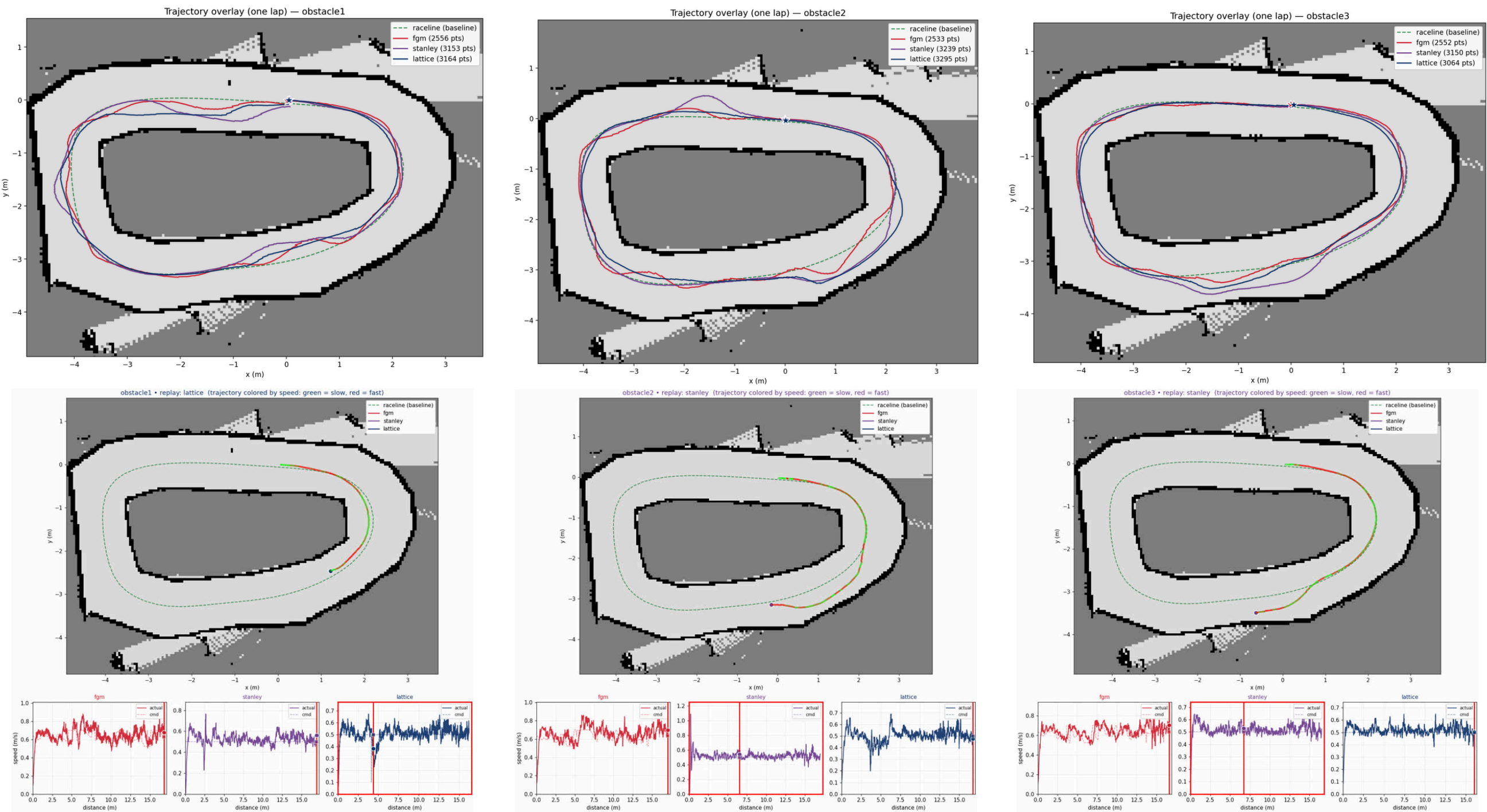
Stanley Avoidance



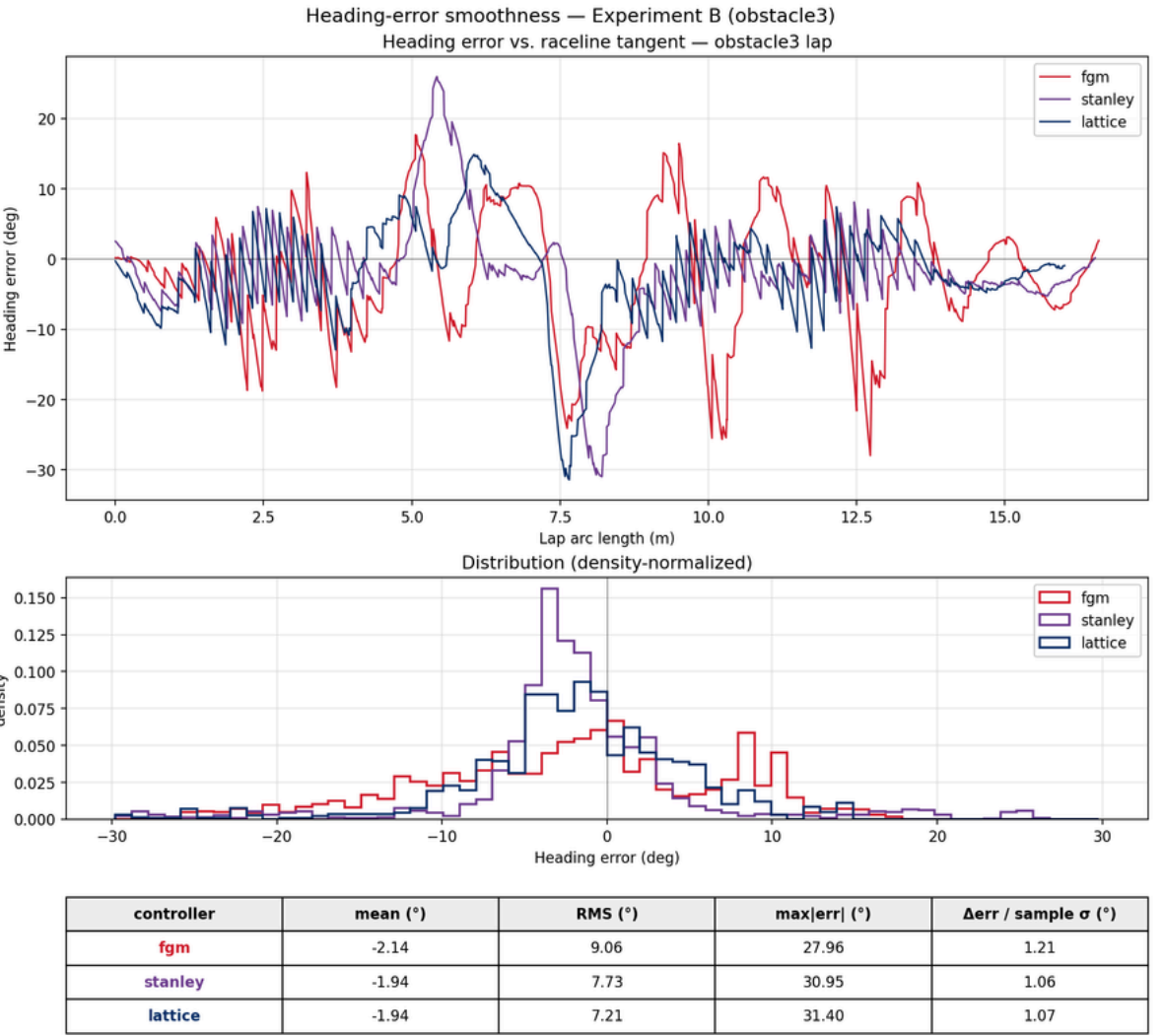
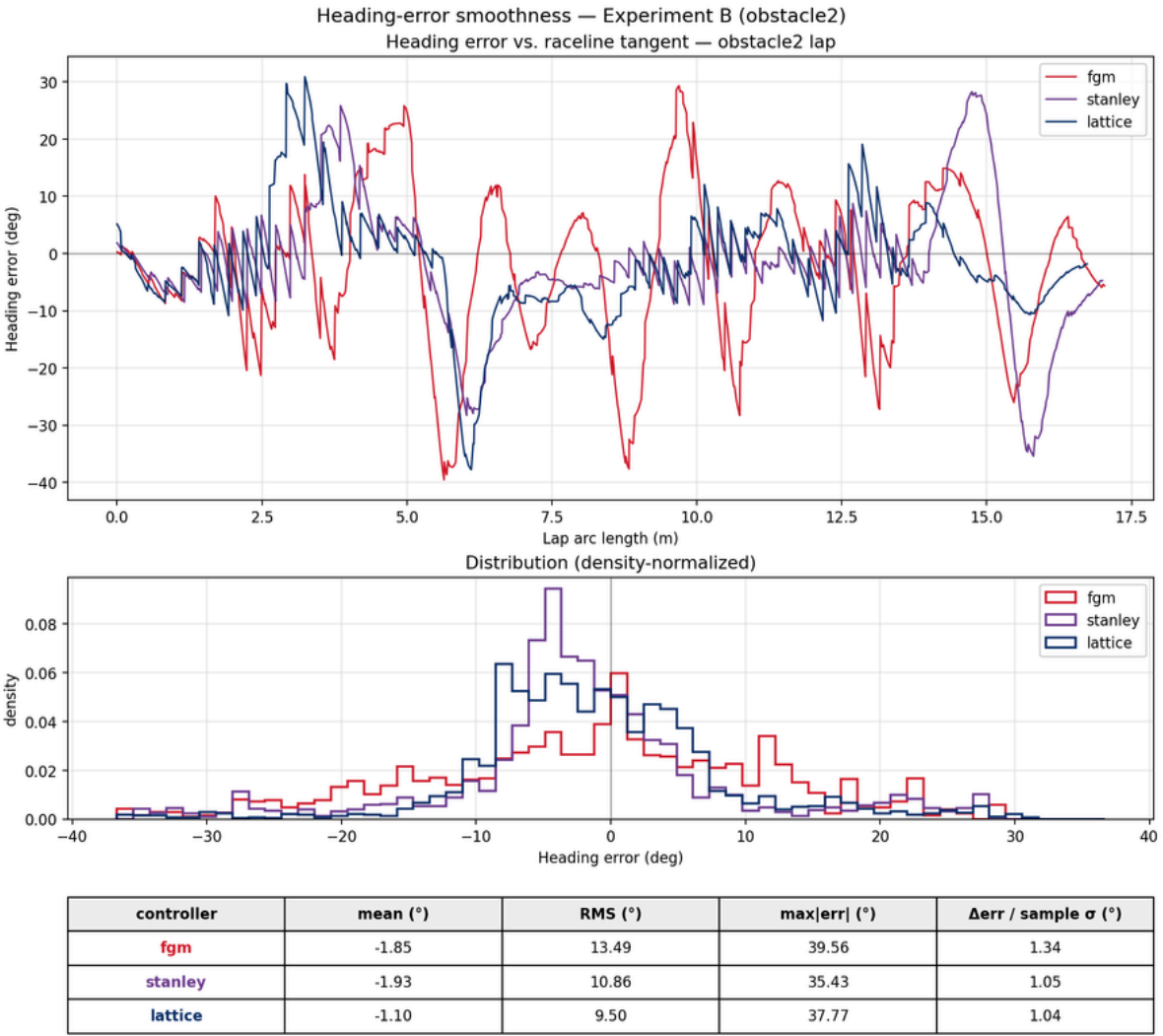
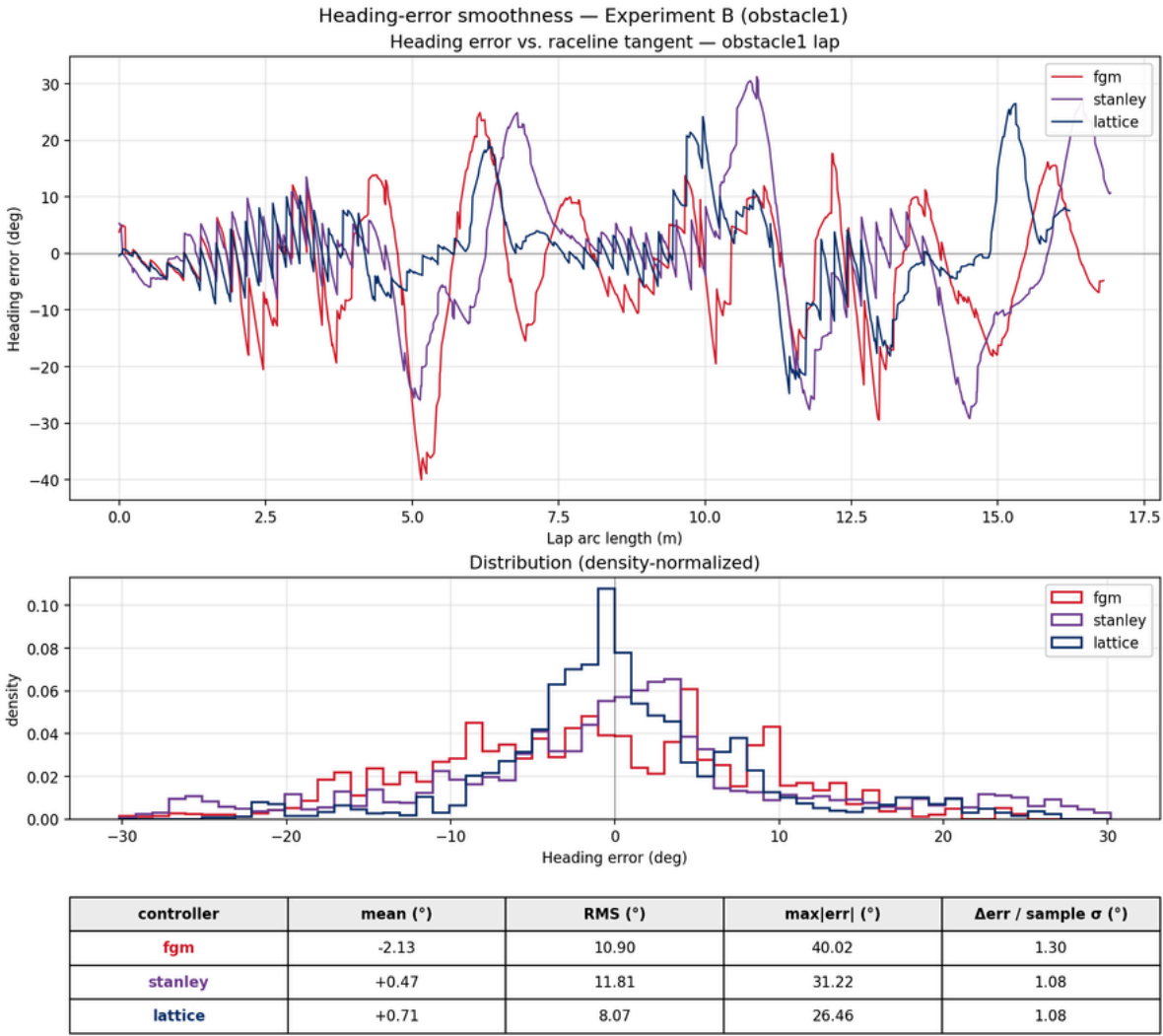
Lattice Planner + Pure Pursuit



Experiment 3 – Static Obstacle Avoidance



Experiment 3 – Static Obstacle Avoidance

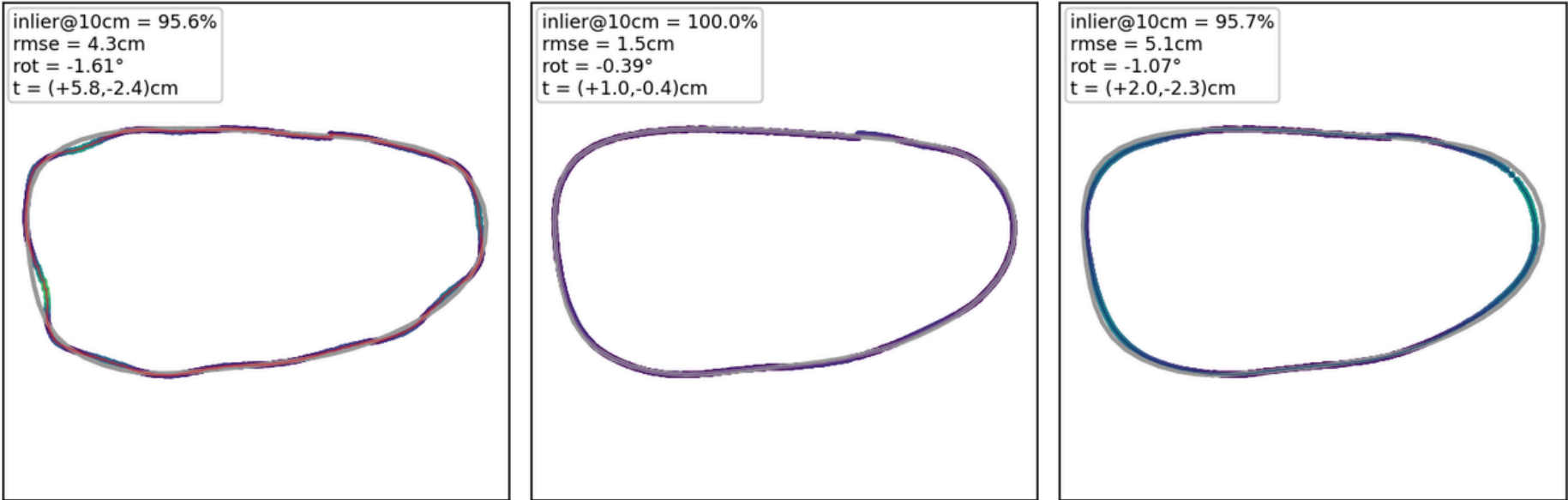




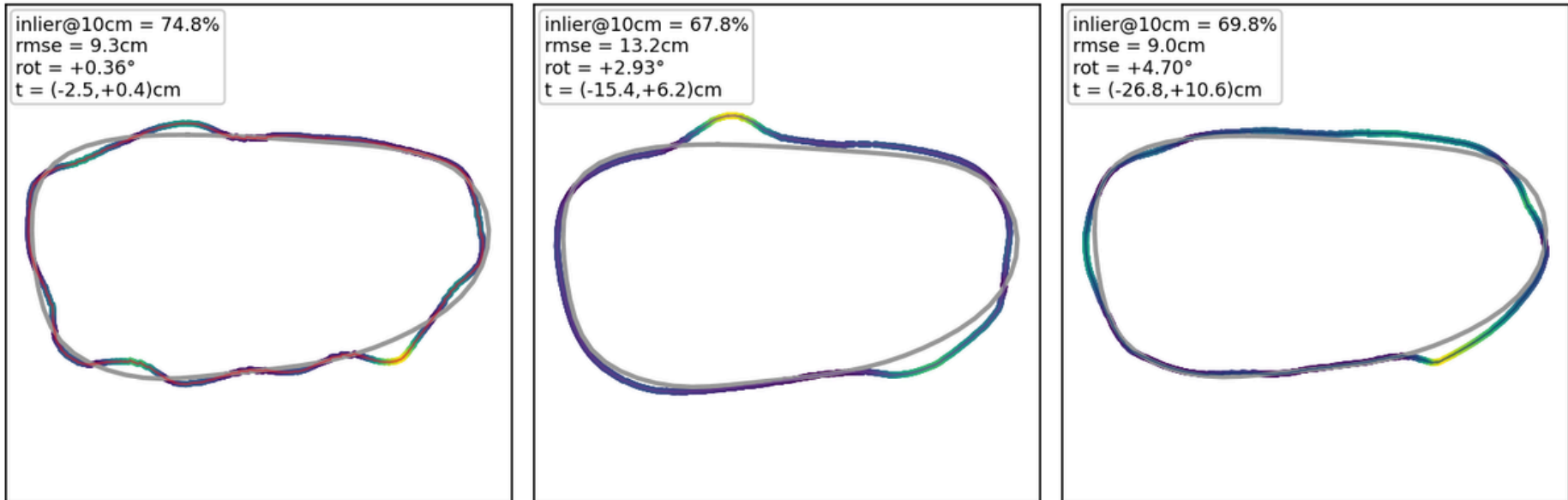
THANK YOU



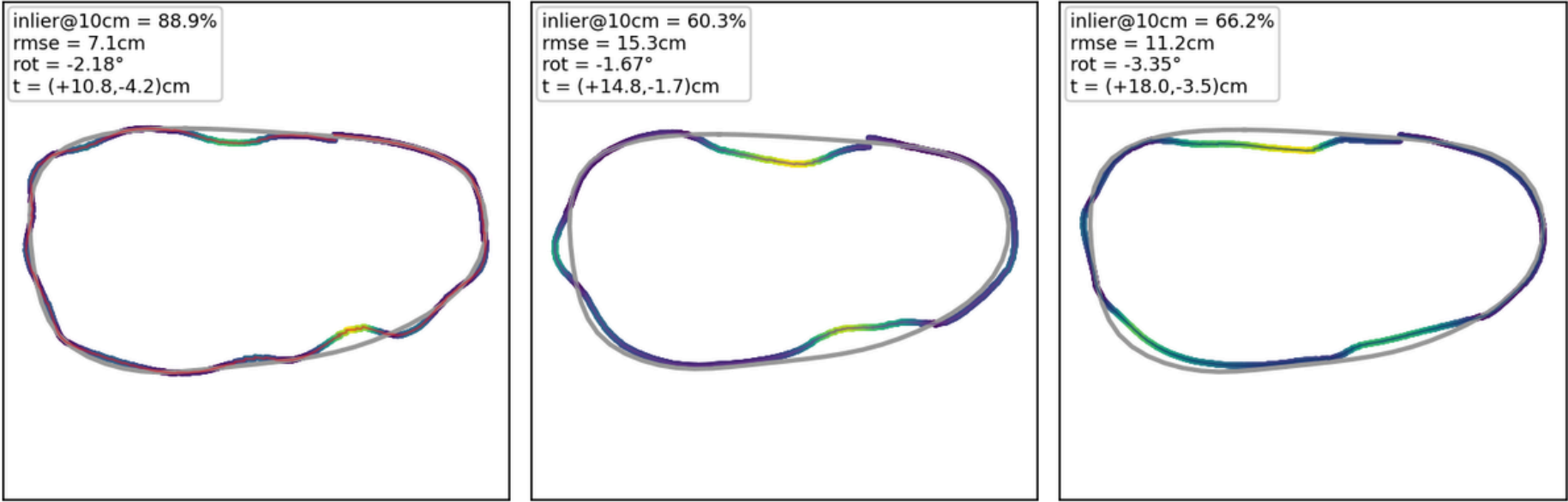
noobs



obstacle2



obstacle1



obstacle3

